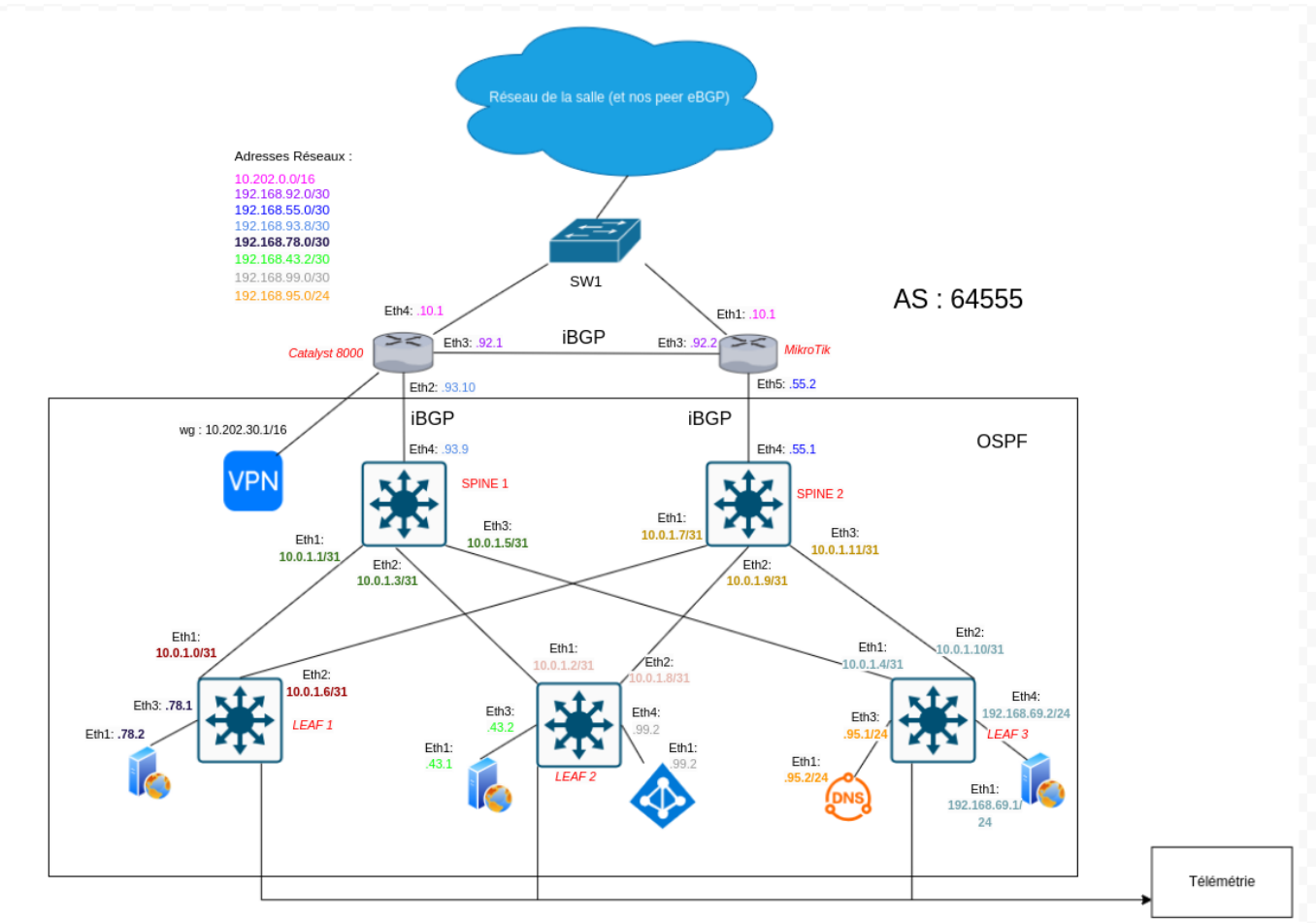


SAE

L'objectif de cette SAE est de concevoir, déployer et valider une architecture réseau de type "leaf & spine" dans un environnement de datacenter simulé, en combinant des équipements physiques et virtualisés (Catalyst 8000, Mikrotik, Switch, routeurs containerisés via Containerlab). Notre mission consiste à assurer l'interconnexion, la haute disponibilité, la sécurité et la supervision des services (web, DNS, monitoring), tout en garantissant l'interopérabilité avec d'autres partenaires via BGP et un VPN Wireguard. L'ensemble du projet s'appuie sur une gestion en mode Kanban, avec validation régulière des jalons par le BBP, et une documentation complète des choix techniques et de la répartition des tâches au sein du groupe.

Schéma Représentatif



Plan d'adressage

Voici le plan d'adressage retenu pour l'ensemble des équipements de notre architecture.

Spine1

Equipement	Interface	Adresse IP	Masque	Description/Peer
------------	-----------	------------	--------	------------------

Equipement	Interface	Adresse IP	Masque	Description/Peer
spine01	Ethernet1	10.0.1.1	255.255.255.254	to-leaf01
spine01	Ethernet2	10.0.1.3	255.255.255.254	to-leaf02
spine01	Ethernet3	10.0.1.5	255.255.255.254	to-leaf03
spine01	Ethernet4	192.168.93.9	255.255.255.252	to-catalyst
spine01	Loopback0	10.255.0.1	255.255.255.255	Loopback

Spine2

Equipement	Interface	Adresse IP	Masque	Description/Peer
spine02	Ethernet1	10.0.1.7	255.255.255.254	to-leaf01
spine02	Ethernet2	10.0.1.9	255.255.255.254	to-leaf02
spine02	Ethernet3	10.0.1.11	255.255.255.254	to-leaf03
spine02	Ethernet4	192.168.55.1	255.255.255.252	to-mikrotik
spine02	Loopback0	10.255.0.2	255.255.255.255	Loopback

Leaf1

Equipement	Interface	Adresse IP	Masque	Description/Peer
leaf01	Ethernet1	10.0.1.0	255.255.255.254	to-spine01
leaf01	Ethernet2	10.0.1.6	255.255.255.254	to-spine02
leaf01	Ethernet3	192.168.94.1	255.255.255.252	to-yokoso-web
leaf01	Ethernet4	192.168.95.1	255.255.255.0	to-yokoso-dns
leaf01	Loopback0	10.255.0.11	255.255.255.255	Loopback

Leaf2

Equipement	Interface	Adresse IP	Masque	Description/Peer
leaf02	Ethernet1	10.0.1.2	255.255.255.254	to-spine01
leaf02	Ethernet2	10.0.1.8	255.255.255.254	to-spine02
leaf02	Loopback0	10.255.0.12	255.255.255.255	Loopback

Leaf3

Equipement	Interface	Adresse IP	Masque	Description/Peer
leaf03	Ethernet1	10.0.1.4	255.255.255.254	to-spine01

Equipement	Interface	Adresse IP	Masque	Description/Peer
leaf03	Ethernet2	10.0.1.10	255.255.255.254	to-spine02
leaf03	Ethernet3	192.168.91.60	255.255.255.0	to-wg-easy
leaf03	Loopback0	10.255.0.13	255.255.255.255	Loopback
leaf03	Ethernet4	10.202.9.2	255.255.255.0	to-PC-physique (optionnel)

Catalyst

Equipement	Interface	Adresse IP	Masque	Description/Peer
catalyst	Ethernet1	192.168.93.10	255.255.255.252	to-spine01
catalyst	Loopback0	10.255.0.100	255.255.255.255	Loopback

Mikrotik

Equipement	Interface	Adresse IP	Masque	Description/Peer
mikrotik	Ethernet1	192.168.55.2	255.255.255.252	to-spine02
mikrotik	Loopback0	10.255.0.200	255.255.255.255	Loopback

wg-easy

Equipement	Interface	Adresse IP	Masque	Description/Peer
wg-easy	eth1	192.168.91.61	255.255.255.0	to-leaf03

web

Equipement	Interface	Adresse IP	Masque	Description/Peer
yokoso-web	eth1	192.168.94.2	255.255.255.252	to-leaf01
yokoso-web2	eth1	192.168.43.2	255.255.255.252	to-leaf02
yokoso-web3	eth1	192.168.69.2	255.255.255.252	to-leaf02

HAProxy

Equipement	Interface	Adresse IP	Masque	Description/Peer
haproxy	eth1	192.168.73.2	255.255.255.0	to-leaf02

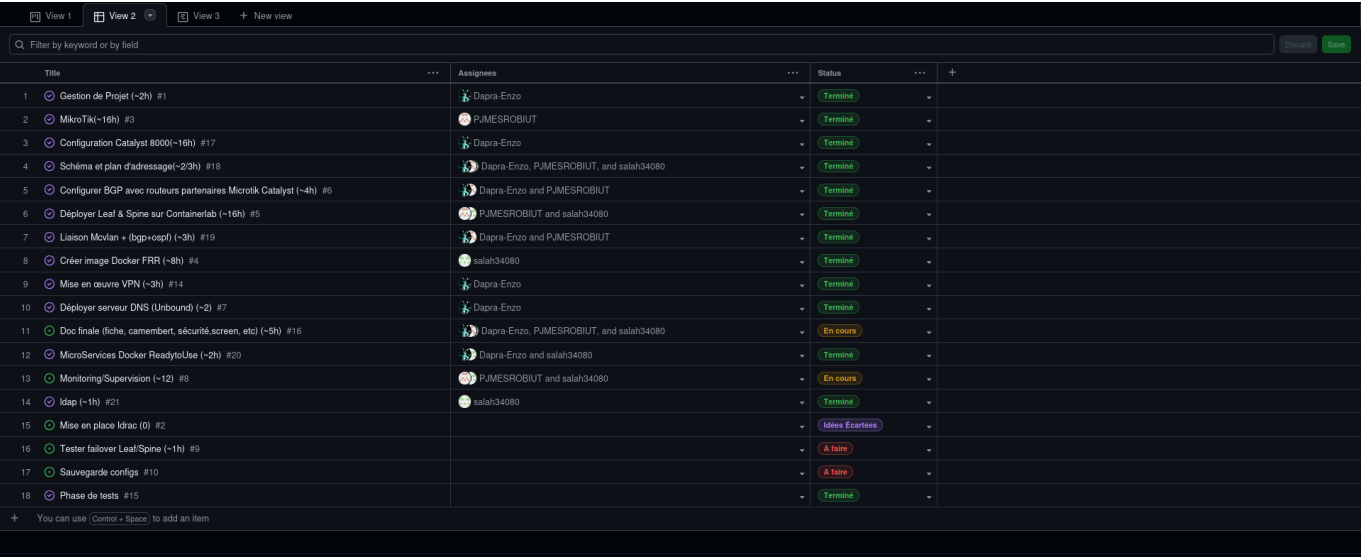
dns

Equipement	Interface	Adresse IP	Masque	Description/Peer
yokoso-dns	eth1	192.168.95.2	255.255.255.0	to-leaf01

Gestion de Projet : utilisation de la méthode Kanban sur GitHub

Pour assurer un suivi rigoureux et transparent de l'avancement de notre projet, nous avons choisi d'utiliser la fonctionnalité **Projets** de GitHub, et plus particulièrement le modèle **Kanban** (voir image ci-dessous image2). Ce choix nous a permis d'organiser efficacement les tâches, de répartir le travail entre les membres de l'équipe, et de garantir une validation régulière des jalons par le BBP (Big Boss Pouchou), conformément aux exigences du sujet.

Le tableau Kanban, accessible via l'onglet "Projects" du dépôt GitHub de notre groupe, se compose de plusieurs colonnes :



Title	Assignees	Status
1 Gestion de Projet (~2h) #1	Diapra-Enzo	Terminé
2 MikroTik (~16h) #3	PJMESROBIUT	Terminé
3 Configuration Catalyst 8000 (~16h) #17	Diapra-Enzo	Terminé
4 Schéma et plan d'adressage (~23h) #18	Diapra-Enzo, PJMESROBIUT, and salah34080	Terminé
5 Configurer BGP avec routeurs partenaires Microtik Catalyst (~4h) #6	Diapra-Enzo and PJMESROBIUT	Terminé
6 Déployer Leaf & Spine sur Containerlab (~16h) #5	PJMESROBIUT and salah34080	Terminé
7 Liaison Mcvlan + (bgp+ospf) (~3h) #19	Diapra-Enzo and PJMESROBIUT	Terminé
8 Créer image Docker FRR (~8h) #4	salah34080	Terminé
9 Mise en œuvre VPN (~3h) #14	Diapra-Enzo	Terminé
10 Déployer serveur DNS (Unbound) (~2) #7	Diapra-Enzo	Terminé
11 Doc finale (fiche, camembert, sécurité, screen, etc) (~9h) #16	Diapra-Enzo, PJMESROBIUT, and salah34080	En cours
12 MicroServices Docker ReadyToUse (~2h) #20	Diapra-Enzo and salah34080	Terminé
13 Monitoring/Supervision (~12) #8	PJMESROBIUT and salah34080	En cours
14 Idap (~1h) #21	salah34080	Terminé
15 Mise en place Idrac (0) #2		Idées Écartées
16 Tester failover Leaf/Spine (~1h) #9		À faire
17 Sauvegarde configs #10		À faire
18 Phase de tests #15		Terminé

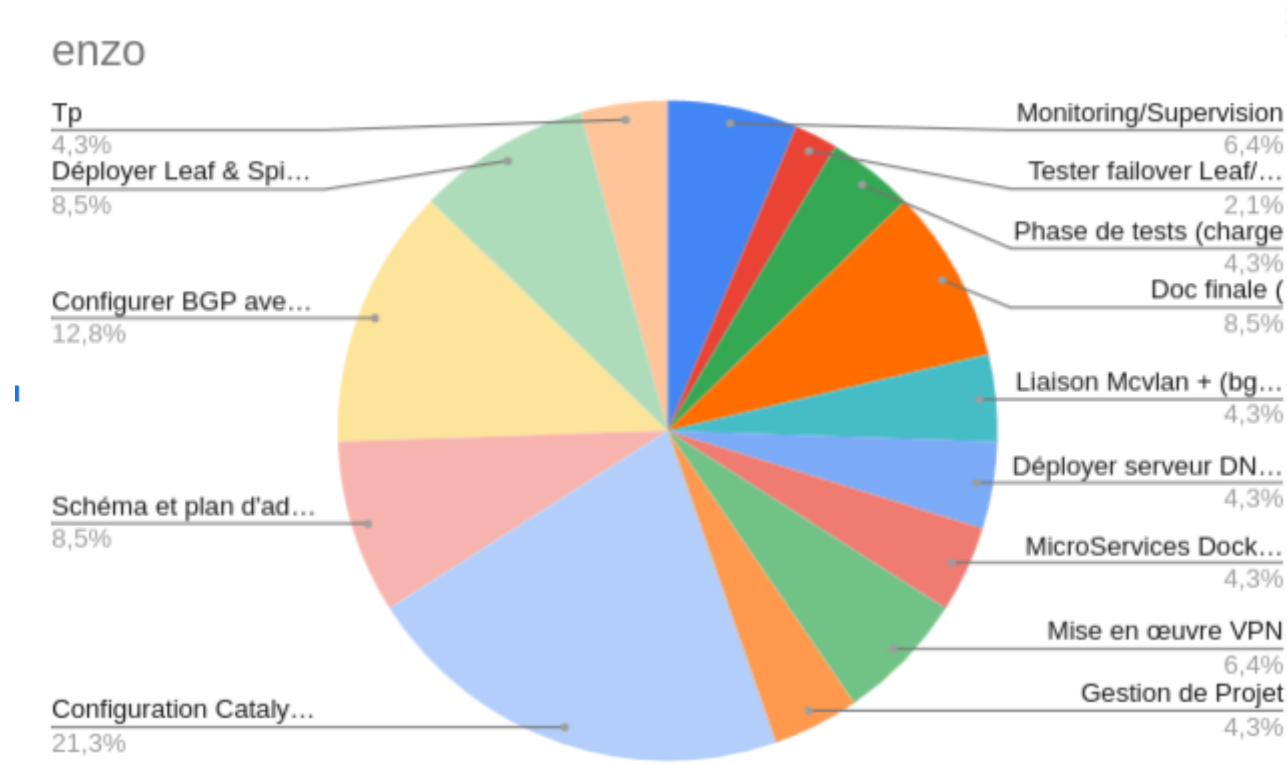
- **À faire** : toutes les tâches identifiées mais non commencées
- **En cours** : tâches en cours de réalisation
- **Terminé** : tâches finalisées
- **Idées Écartées** : éléments jugés non pertinents ou mis de côté

Répartition et suivi des tâches par membre

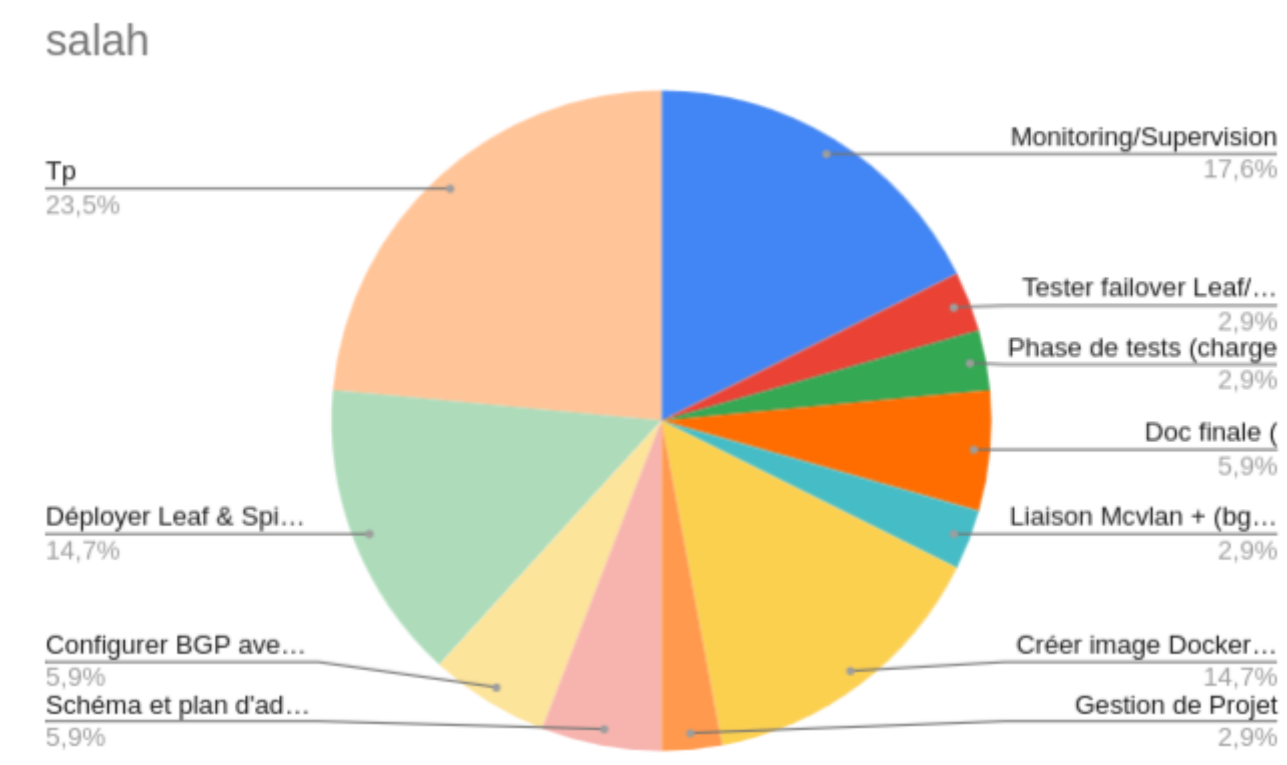
Détails horaires

Retrouvez ci-dessous la répartition synthétique des tâches réalisées par chaque membre.
Pour un détail complet des tickets et du suivi, le fichier CSV est disponible [ici](#) dans le dépôt GitHub.

Enzo

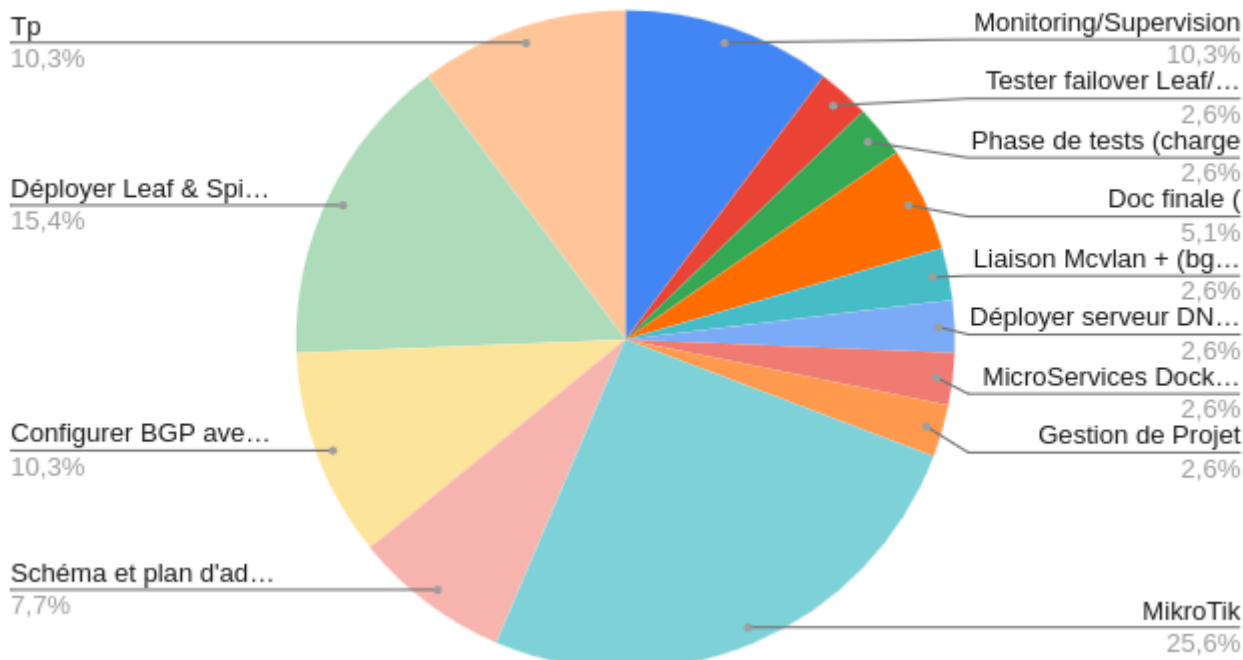


Salah



Pierre

pierre



Choix des Technos + Plan d'adressage avec son schéma

Pour répondre efficacement aux besoins du projet, nous avons soigneusement sélectionné chaque technologie et protocole en fonction de leur pertinence, compatibilité et facilité de prise en main.

Routage dynamique

- **iBGP** a été utilisé entre le Catalyst et MikroTik, entre Catalyst et Spine1, ainsi qu'entre MikroTik et Spine2.

More actions

Pourquoi ce choix ?

- L'iBGP permet une gestion fine des routes au sein de notre AS privé (AS64555).
- Il offre un contrôle précis sur la distribution des routes et facilite la redondance.
- Cela simplifie l'interconnexion entre équipements "core" tout en gardant la maîtrise sur les annonces.

- **OSPF** est employé pour le reste du réseau.

Avantages :

- **OSPF** est employé pour le reste du réseau.

Avantages :

- OSPF est rapide à converger et parfaitement adapté pour la diffusion automatique des routes dans une topologie leaf & spine.
- Il limite les configurations manuelles et s'intègre bien avec les équipements virtuels comme Arista.

- **En résumé** : L'association iBGP (cœur du réseau, contrôle) + OSPF (distribution automatique, rapidité) permet d'avoir un routage efficace, souple et facile à superviser.
-

Orchestration & virtualisation

- **Containerlab**

Nous avons choisi Containerlab plutôt que Netlab, car il offre une gestion complète, transparente et documentée de la topologie réseau.

Netlab a tendance à “faire de la magie” et à masquer certains détails, alors que Containerlab permet de mieux comprendre et maîtriser chaque lien, chaque conteneur et chaque configuration réseau.

- **Arista** (pour les spines et leafs)

Nous avons privilégié les images Arista, car nous connaissons déjà bien cet environnement.

- Compatibilité supérieure avec les outils de monitoring, le routage dynamique et les scénarios multi-vendeurs (mieux que FRRouting dans ce contexte).
- Interface CLI familière et documentation claire.

- **macvlan**

Utilisé pour assurer la liaison entre le monde physique (Catalyst, MikroTik) et le virtuel (Containerlab), via les spines.

- Permet une intégration transparente entre les conteneurs et les équipements physiques, sans NAT, et une gestion fine des VLAN.
-

Administration et interconnexion

- **Pas d'iDRAC**

Nous n'avons pas retenu iDRAC, car dans notre cas, tous les accès se font en liaison directe et nous n'avons pas besoin d'administration distante avancée.

- *Avantages d'iDRAC* : gestion serveur à distance, accès en cas de crash système.
- *Inconvénients ici* : complexité supplémentaire, inutile dans une maquette “tout direct”, moins plug-and-play.
- *Avantage de notre choix* : simplicité, moins de points de panne, déploiement rapide.

- **AS64555**

Utilisé pour rester dans la plage des AS privés (conforme aux bonnes pratiques pour les projets étudiants).

VPN et sécurité

- **Wireguard Easy**

Nous avons choisi Wireguard Easy pour sa simplicité de déploiement, la génération et distribution automatique des clés, ainsi que la présence d'une interface web intuitive pour la gestion des pairs et des configurations.

Supervision et monitoring

- **Uptime Kuma**

Nous avons sélectionné Uptime Kuma car c'est un outil de monitoring léger, visuel et facile à mettre en place.

Il permet de superviser simplement la disponibilité des services critiques (web, DNS, VPN...) avec alertes en cas d'incident.

- **Oxydyse**

Outil utilisé pour enregistrer automatiquement les "show run" de nos équipements réseau, assurant ainsi un archivage fiable des configurations et facilitant la traçabilité et la restauration en cas de besoin.

- **Grafana + GNMIC + Prometheus**

- **Prometheus** collecte les métriques réseau de façon centralisée.
- **GNMIC** récupère les métriques via gNMI sur les équipements compatibles (notamment Arista).
- **Grafana** offre un tableau de bord moderne, personnalisable et interactif pour visualiser en temps réel l'état du réseau, les performances et anticiper d'éventuels problèmes.

Déploiement et portabilité

- **Docker packages avec dépôt personnel (ghcr.io)**

Nous avons packagé nos microservices dans des images Docker stockées sur le registre GitHub (ghcr.io), ce qui permet un déploiement rapide et reproductible, même en cas de changement de machine.

Les images sont "ready to use" pour Containerlab ou tout autre orchestrateur Docker.

En résumé :

Ces choix nous ont permis de bâtir une infrastructure réseau complète, fiable, facilement supervisable et parfaitement adaptée à un contexte pédagogique et d'expérimentation.

Configuration du Catalyst 8000

Prérequis

Installation de picocom sur la machine :

```
apt install picocom
```

Recherche du TTY :

```
dmesg | grep tty
```

Notre tty est le suivant : `/dev/ttyS0`

Connexion série :

```
sudo picocom -b 9600 /dev/ttyS0
```

Appuyer sur entrée puis se connecter :

- login : `admin`
- mdp : `IutB2025?`
- premier mdp a modifier : `Admin123#`

Configuration initiale du Catalyst pour BGP

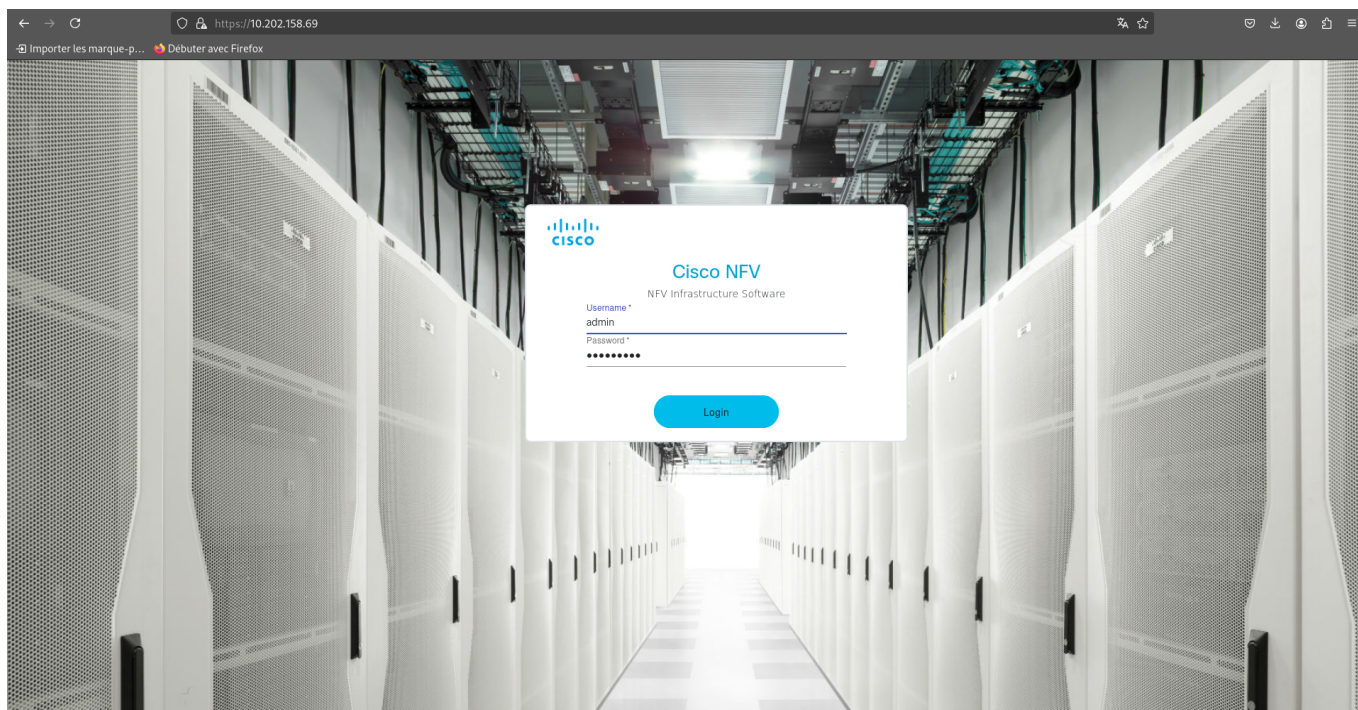
```
enable
configure terminal
nfvis(config)# system settings mgmt ip address 10.202.158.69 255.255.0.0
nfvis(config)# system settings default-gw 10.202.255.254
nfvis(config)# system settings hostname c8200-1
nfvis# commit
nfvis(config)# end
nfvis# wr
```

Pour désactiver le DHCP en cas de problème sur le Catalyst :

```
nfvis(config)# no system settings wan dhcp
nfvis(config)# no bridges bridge wan-br dhcp
nfvis(config)# commit
Commit complete.
```

Accès à l'interface Web Catalyst

Accéder à l'interface Web en entrant l'IP configurée : **10.202.158.69**



Après connexion, on arrive sur le dashboard :

System Details

System Status:	Healthy
PID:	C8200-UCPE-1N8
Serial Number:	FGL2530M6SS
BIOS-Version:	C8200-UCPE_1.04.103020201614
CIMC-Version:	NA

Resource Utilization

System CPU Utilization:	19.89 %
Storage Utilization:	18.49 %

VM Details

Total VM:	0
Active VM:	0
Failed VM:	0
Shutdown VM:	0
Transitioning VM:	0

Resource Allocation

Total Cores:	8	Total Memory:	15.54 GB
System Occupied:	1	System Memory:	3.5390625 GB
Allocated Cores:	0	Allocated Memory:	0 GB
Allocated %:	12.50%	Allocated %:	22.77%

Interface Status

GE0-0:	down
GE0-1:	down
GE0-2:	up
GE0-3:	down
GE0-4:	down
GE0-5:	down

Failed Notifications

No Failed Notifications.

Déploiement d'un Routeur sur le Catalyst

1. Récupérer le fichier sur Nextcloud :

`c8000v-universalk9_16G_serial.17.06.01a.tar.gz`

2. Ajout de l'image dans l'interface :

Aller dans **Configuration** → **Virtual Machine** → **images** → **Images Repository**

NFVIS

CR200-UCPE-1N8 NFVIS-4.8.1-FC1 c8200

admin

Administrator

Thu Mar 06, 07:29:35 (anon)

Dashboard

Configuration

Monitoring

Operations

Platform

Notification

Images

Image upload and information

Select File (.iso,.ova,.tgz,.tar,.gz,.img,.vmdk,.qcow2,.raw)

c8000v-universalk9_16G_serial.17.06.01a.tar.gz

Images

Total Record: 0

search in all record

#	Image Name	State	Type	Version	Placement	Secure Boot	Action
---	------------	-------	------	---------	-----------	-------------	--------

Page 1 of 0

Go to Page: 1

Show 10 rows

Profiles

Flavor Information

Profiles

Total Record: 0

search in all record

#	Profile	CPU	Memory(MB)	Disk(MB)	Sockets	Cores	Threads	Source Image	Action
---	---------	-----	------------	----------	---------	-------	---------	--------------	--------

Page 1 of 0

Go to Page: 1

Show 10 rows

3. Création du profil pour le routeur :

Add Profile

Profile *

RouterC8200

CPU *

2

1-32

Memory(MB) *

4096

256-262144

Disk(GB) *

4

1-1000

Sockets

1

Cores

2

Threads

1

Source Image

c8000v-universalk9_16G_serial.17.06.01a.tar.gz

Submit

Cancel

Reset

Grâce au tar.gz, des profils “petit”, “moyen” et “grand” sont créés automatiquement selon les besoins.

Profiles

Flavor Information

+

Profiles

Total Record: 4

search in all record

#	Profile	CPU	Memory(MB)	Disk(MB)	Sockets	Cores	Threads	Source Image
1	C8000V-medium	4	4096	8192				c8000v-universalk9_16G_serial.17.06.01a.tar.gz
2	C8000V-mini	1	4096	8192				c8000v-universalk9_16G_serial.17.06.01a.tar.gz
3	C8000V-small	2	4096	8192				c8000v-universalk9_16G_serial.17.06.01a.tar.gz
4	RouterC8200	2	4096	4096	1	2	1	c8000v-universalk9_16G_serial.17.06.01a.tar.gz

<<<1>>>

Page 1 of 1 | Go to Page: 1 | Show 10 rows

4. Déploiement de la VM routeur via l'interface graphique :

OTHER NETWORK

ROUTER7

GE0-1

GE0-3

GE0-4

GE0-5

IMPORT VM

VM Details

VM Name *
ROUTER7

Image *
c8000v-universalk9_16G_serial.17.06.01a.tar.gz

Profile *
C8000V-small

Group Name

VNC Password

CONTROLLER *
non-vManage

TECH_PACKAGE *
network-essentials

NGIO *
ENABLE

CRYPTO_OFFLOAD *
ENABLE

SSH_USERNAME *

SSH_PASSWORD *

Port Number

DEPLOY

Attendre quelques dizaines de secondes que la VM se déploie, puis vérifier sa présence dans **VM Manage** :



The screenshot shows the Nfvis web interface. At the top, there's a navigation bar with links like Dashboard, Configuration, Monitoring, Operations, Platform, and Notification. Below this is the 'Manage VM' section. A table titled 'VM List' displays the following data:

#	Name	Status	Flavor	Group	Port Forwarding	vnic0	vnic1	vnic2	vnic3	vnic4	vnic5	vnic6	vnic7	vnic8	vnic9	vnic10	vnic11	Management IP	Action
1	ROUTER7	Booting	C8000V-small		2222=>22	Internal												10.202.158.70	

At the bottom right of the table, it says 'Page 1 of 1 | Go to Page: 1 | Show 10 rows'.

NB : Mot de passe VM administrateur : `AdmininstrateurIutB2025TTadmin_@@@`

Connexion et configuration du routeur

1. Accès console au routeur "ROUTER7"

Depuis la CLI nfvis :

```
vmConsole ROUTER7
```

2. Passer les interfaces en UP

```
no shutdown
```

3. Affecter les adresses IP

- **Sortie salle Ge0/2 et 4 (virt) :**

```
ip address 10.202.10.1 255.255.0.0
no shut
```

- **Côté Containerlab Ge0/0 et 2 (virt) :**

```
ip address 192.168.93.10 255.255.255.252
ip address 192.168.254.1 255.255.255.252
no shut
```

Configuration BGP et voisins

```
router bgp 64555
neighbor 10.202.159.250 remote-as 100          // Catalyst Ng Pao Mathias
Ludovic
neighbor 10.202.159.251 remote-as 100          // Mikrotik Ng Pao Mathias
Ludovic
neighbor 10.202.1.11 remote-as 1713            // Romain Nathan
neighbor 10.202.1.50 remote-as 1713            // Romain Nathan
neighbor 192.168.254.2 remote-as 64555         // ibgp
```

Vérification

Afin de vérifier le bon fonctionnement de mon switch Catalyst et la connectivité réseau globale, j'ai réalisé une série de tests techniques : Vérification de la configuration BGP J'ai utilisé la commande suivante pour afficher la section BGP de la configuration du routeur :

```
Router#sh running-config | section bgp
router bgp 64555
  bgp log-neighbor-changes
  network 10.202.0.0 mask 255.255.0.0
  neighbor 10.202.1.11 remote-as 1713
  neighbor 10.202.1.50 remote-as 1713
  neighbor 10.202.159.250 remote-as 100
  neighbor 10.202.159.251 remote-as 100
  neighbor 10.202.214.1 remote-as 213661
  neighbor 10.202.214.2 remote-as 213661
  neighbor 10.255.0.1 remote-as 64555
  neighbor 10.255.0.1 update-source Loopback0
  neighbor 10.255.0.254 remote-as 64555
  neighbor 10.255.0.254 update-source Loopback0
  neighbor 192.168.92.2 remote-as 64555
  neighbor 192.168.93.9 remote-as 64555
```

Vérification de l'état des sessions BGP:

```
Router#sh ip bgp summary
BGP router identifier 192.168.254.1, local AS number 64555
BGP table version is 7937, main routing table version 7937
105 network entries using 26040 bytes of memory
318 path entries using 43248 bytes of memory
32/13 BGP path/bestpath attribute entries using 9472 bytes of memory
3 BGP AS-PATH entries using 272 bytes of memory
0 BGP route-map cache entries using 0 bytes of memory
0 BGP filter-list cache entries using 0 bytes of memory
BGP using 79032 total bytes of memory
BGP activity 1421/1316 prefixes, 7034/6716 paths, scan interval 60 secs
105 networks peaked at 12:31:22 Jun 13 2025 UTC (00:57:10.280 ago)
```

Neighbor	V	AS	MsgRcvd	MsgSent	TblVer	InQ	OutQ	Up/Down	State/PfxRcd
10.202.1.11	4	1713	6002	5811	7937	0	0	3d04h	22
10.202.1.50	4	1713	3155	2420	7937	0	0	1d06h	22
10.202.159.250	4	100	2554	2471	7937	0	0	1d07h	68
10.202.159.251	4	100	340777	2334	7937	0	0	1d05h	56
10.202.214.1	4	213661	745	507	7937	0	0	05:36:39	22

Neighbor	V	AS	MsgRcvd	MsgSent	TblVer	InQ	OutQ	Up/Down	State/PfxRcd
10.202.214.2	4	213661	2538	2316	7937	0	0	1d05h	22
10.255.0.1	4	64555	0	0	1	0	0	2d01h	Active
10.255.0.254	4	64555	0	0	1	0	0	never	Active
192.168.92.2	4	64555	338981	2488	7937	0	0	1d05h	91
192.168.93.9	4	64555	259	388	7937	0	0	03:30:38	14

Cette commande m'a permis de m'assurer que les sessions BGP étaient bien établies avec les voisins attendus.

Tests de connectivité par ping Pour confirmer l'accessibilité des services interconnectés, j'ai effectué plusieurs pings :

```
Router#ping 1.0.90.106
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 1.0.90.106, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/1/3 ms
Router#ping 192.168.78.2
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.78.2, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/1/3 ms
Router#ping 192.168.95.2
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.95.2, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/1/2 ms
Router#ping 10.202.30.1
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.202.30.1, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/2/6 ms
Router#
```

Premier ping : vers un service hébergé par un autre groupe, pour valider la connectivité inter-groupe.

Deuxième ping : vers le serveur HAProxy, afin de m'assurer que le répartiteur de charge était joignable.

Troisième ping : vers le serveur DNS, pour vérifier la résolution de noms.

Quatrième ping : vers le service VPN, pour confirmer l'accès sécurisé via le tunnel. Tests de routage avec traceroute

```
Router#traceroute 192.168.95.2
Type escape sequence to abort.
Tracing the route to 192.168.95.2
VRF info: (vrf in name/id, vrf out name/id)
 1 192.168.93.9 1 msec 1 msec 1 msec
 2 10.0.1.0 2 msec 2 msec 1 msec
 3 192.168.95.2 1 msec 2 msec 2 msec
Router#traceroute 1.0.90.102
Type escape sequence to abort.
Tracing the route to 1.0.90.102
VRF info: (vrf in name/id, vrf out name/id)
 1 10.202.214.1 1 msec 1 msec 0 msec
 2 172.16.34.3 2 msec 1 msec 2 msec
 3 1.0.10.10 2 msec 1 msec 1 msec
 4 1.0.90.102 [AS 213661] 2 msec 2 msec 2 msec
```

Traceroute vers notre propre serveur DNS : ce test a permis de visualiser le chemin emprunté par les paquets jusqu'à notre DNS interne et de vérifier qu'il n'y avait pas de coupure ou de latence excessive.

Traceroute vers le serveur DNS du groupe Triforce : ce test a validé l'accessibilité entre groupes via BGP, et a permis d'observer le passage par les routeurs Spine/Leaf configurés.

L'ensemble de ces tests a permis de valider le bon fonctionnement du Catalyst et la connectivité des services dans l'infrastructure.

Divers

- **Reset Catalyst (hors images) :**

```
factory-default-reset all-except-images
```

Questions / Problèmes rencontrés

- **Question** : Comment ça se fait que quand j'ajoute un management sur mon iutbrt2024 [...] ?
- **À compléter selon les retours d'expérience ou questions supplémentaires.**

Configuration du MikroTik

1. Préparation de l'équipement

Sur le MikroTik, on a supprimé tous les bridges, à part l'interface loopback qui reste en bridge.
Toutes les autres interfaces Ethernet (ether1, ether2, etc.) fonctionnent en mode "normal" (pas en bridge).
De cette manière, on aura pas de problème de switch sur nos ports:

		Name	Type	Actual MTU
- D	R	 Loopback	Bridge	1500
D	R	 ether1	Ethernet	1500
D		 ether2	Ethernet	1500
D	R	 ether3	Ethernet	1500
D		 ether4	Ethernet	1500
D	R	 ether5	Ethernet	1500

2. Attribution des adresses IP

Nous avons ensuite configuré les adresses IP sur les différents ports :

- Un lien vers la salle : IP **10.202.20.1/16** sur **ether1**
- Un lien vers le Catalyst : IP **192.168.92.2/30** sur **ether3**
- Un lien vers le Spine 2 : IP **192.168.55.2/30** sur **ether5**
- Une IP de loopback : **10.255.0.254/32**
- Un port de sécurité : IP **192.168.1.1/24** sur **ether4** (pour avoir un accès en cas de perte des autres IP pour configurer le mikrotik)

::: Lien vers la salle				
- D		 10.202.20.1/16	10.202.0.0	ether1
::: loopback				
- D		 10.255.0.254/32	10.255.0.254	Loopback
::: Port Sécurité (au cas où)				
- D		 192.168.1.1/24	192.168.1.0	ether4
::: Lien vers spine2				
- D		 192.168.55.2/30	192.168.55.0	ether5
::: Lien vers catalyst				
- D		 192.168.92.2/30	192.168.92.0	ether3

3. Configuration du DHCP

Nous avons laissé le serveur DHCP actif sur le port de sécurité (ether4) pour faciliter l'accès en cas de besoin :

DHCP								
NetworksLeasesOptionsOption SetsVendor ClassesAlerts								
Add NewDHCP ConfigDHCP Setup								
1 item								
		Name	Interface	Relay	Lease Time	Address Pool	Add ARP For Leases	
- D	/	dhcp1	ether4		00:10:00	dhcp_pool1	no	

Il est grisé sur l'image car il n'y a aucun appareil qui utilise le service.

4. Mise en place du BGP

Pour faire du routage dynamique avec nos collègues, les spines et le Catalyst, nous avons configuré BGP sur le Mikrotik.

Nous avons déclaré nos voisins, renseigné l'adresse distante ainsi que l'AS distante.

Voici ici tout nos peers, qui sont a l'état **established** :

	Name	Instance	Remote Address	Remote AS	M...	R... R...	TTL	Remote ID	Uptime	Prefix Count	State
- D	Cata_Nathan	default	10.202.1.11	1713	no	no	default	10.17.1.1	22:23:57	9	established
- D	Cata_triforce	default	10.202.214.2	213661	no	no	default	1.1.1.1	04:32:53	9	established
- D	Catalyst-Yokoso	default	192.168.92.1	64555	no	no	default	192.168.254.1	04:02:59	19	established
- D	Catalyst_Mathias	default	10.202.159.250	100	no	no	default	55.55.55.55	04:40:59	10	established
- D	Mikro_Nathan	default	10.202.1.50	1713	no	no	default	10.17.10.2	05:47:11	7	established
- D	Mikro_Triforce	default	10.202.214.1	213661	no	no	default	2.2.2.2	04:32:48	9	established
- D	Mikrotik_Mathias	default	10.202.159.251	100	no	no	default	10.202.159.251	04:42:37	10	established
- D	To-Spine2	default	192.168.55.1	64555	no	no	255	10.255.0.2	00:04:48	11	established

Voici ici les **réseaux** que nous **annonçons** par BGP:

Network	
::: Réseau de la salle	
- D	10.202.0.0/16
::: reseau vers spine	
- D	192.168.55.0/30
::: reseau vers catalyst	
- D	192.168.92.0/30

5. Supervision du peering BGP

Dans l'onglet **Advertisements**, nous retrouvons l'ensemble des annonces de routes effectuées par nos voisins :

41 items

	Peer	Prefix	Nexthop	AS Path	Origin	Local Pref.	MED	
	 Catalyst_Mathias	10.0.1.10/31	10.202.20.1		igp	0		
	 Catalyst_Mathias	10.0.0.5/32	10.202.100.1	100,65100	incomplete	0		
	 Catalyst_Mathias	192.168.10.0/24	10.202.100.1	100,65100	incomplete	0		
	 Catalyst_Mathias	172.20.20.0/24	10.202.20.1		igp	0		
	 Catalyst_Mathias	10.17.0.2/31	10.202.1.11	1713	igp	0		
	 Catalyst_Mathias	10.0.0.7/32	10.202.100.1	100,65100	incomplete	0		
	 Catalyst_Mathias	10.17.20.1/32	10.202.1.11	1713	igp	0		
	 Catalyst_Mathias	192.168.90.0/24	10.202.100.1	100,65100	incomplete	0		
	 Catalyst_Mathias	192.168.150.0/24	10.202.100.1	100,65100	igp	0		
	 Catalyst_Mathias	192.168.40.0/24	10.202.100.1	100,65100	incomplete	0		
	 Catalyst_Mathias	10.0.0.2/32	10.202.100.1	100,65100	incomplete	0		
	 Catalyst_Mathias	192.168.92.0/30	10.202.20.1		igp	0		
	 Catalyst_Mathias	192.168.55.0/30	10.202.20.1		igp	0		
	 Catalyst_Mathias	10.0.0.6/32	10.202.100.1	100,65100	incomplete	0		
	 Catalyst_Mathias	192.168.80.0/24	10.202.100.1	100,65100	incomplete	0		
	 Catalyst_Mathias	1.0.90.104/30	10.202.214.2	213661	igp	0		
	 Catalyst_Mathias	10.255.0.2/32	10.202.20.1		igp	0		
	 Catalyst_Mathias	10.0.0.4/32	10.202.100.1	100,65100	incomplete	0		
	 Catalyst_Mathias	192.168.152.0/24	10.202.100.1	100,65100	igp	0		
	 Catalyst_Mathias	10.17.0.8/29	10.202.1.11	1713	inn	0		

Catalyst_Mathias

Cata_Nathan

Cata_triforce

Catalyst-Yokoso

Catalyst_Mathias

Mikro_Nathan

Mikro_Triforce

Mikrotik_Mathias

To-Spine2

all

6. Vérification

Tests de connectivité depuis le MikroTik

Enfin, il est possible de vérifier l'ensemble des routes apprises en ligne de commande avec `/ip route print`, ce qui permet de voir la cohérence du routage, et nous voyons bien les routes :

```
[admin@MikroTik] > /ip route print
Flags: X - disabled, A - active, D - dynamic, C - connect, S - static, r
```

#	DST-ADDRESS	PREF-SRC	GATEWAY	DISTANCE
0	ADb 1.0.70.100/30		10.202.214.2	20
1	Db 1.0.70.100/30		10.202.214.1	20
2	Db 1.0.70.100/30		10.202.214.2	200
3	Db 1.0.70.100/30		10.202.214.1	20
4	Db 1.0.70.100/30		10.202.214.2	20
5	ADb 1.0.80.100/30		10.202.214.2	20
6	Db 1.0.80.100/30		10.202.214.1	20
7	Db 1.0.80.100/30		10.202.214.2	200
8	Db 1.0.80.100/30		10.202.214.2	20
9	Db 1.0.80.100/30		10.202.214.2	20
10	ADb 1.0.80.104/30		10.202.214.2	20
11	Db 1.0.80.104/30		10.202.214.1	20
12	Db 1.0.80.104/30		10.202.214.2	200
13	Db 1.0.80.104/30		10.202.214.2	20
14	Db 1.0.80.104/30		10.202.214.2	20
15	ADb 1.0.90.100/30		10.202.214.2	20
16	Db 1.0.90.100/30		10.202.214.1	20
17	Db 1.0.90.100/30		10.202.214.2	200
18	Db 1.0.90.100/30		10.202.214.2	20
19	Db 1.0.90.100/30		10.202.214.2	20
20	ADb 1.0.90.104/30		10.202.214.2	20
21	Db 1.0.90.104/30		10.202.214.1	20
22	Db 1.0.90.104/30		10.202.214.2	200
23	Db 1.0.90.104/30		10.202.214.2	20
24	Db 1.0.90.104/30		10.202.214.2	20
25	ADb 10.0.0.1/32		10.202.100.1	20
26	Db 10.0.0.1/32		10.202.100.1	20
27	Db 10.0.0.1/32		10.202.100.1	200
28	ADb 10.0.0.2/32		10.202.100.1	20
29	Db 10.0.0.2/32		10.202.100.1	20
30	Db 10.0.0.2/32		10.202.100.1	200
31	ADb 10.0.0.3/32		10.202.100.1	20
32	Db 10.0.0.3/32		10.202.100.1	20
33	Db 10.0.0.3/32		10.202.100.1	200
34	ADb 10.0.0.4/32		10.202.100.1	20
35	Db 10.0.0.4/32		10.202.100.1	20
36	Db 10.0.0.4/32		10.202.100.1	200
37	ADb 10.0.0.5/32		10.202.100.1	20
38	Db 10.0.0.5/32		10.202.100.1	20
39	Db 10.0.0.5/32		10.202.100.1	200
40	ADb 10.0.0.6/32		10.202.100.1	20

```
-- [Q quit|D dump|down]
```

Pour valider l'intégration du routeur MikroTik dans l'infrastructure réseau et vérifier le bon fonctionnement des protocoles de routage, plusieurs tests ont été réalisés depuis ce routeur :

```
[admin@MikroTik] > ping 1.0.90.106
SEQ HOST                                SIZE TTL TIME  STATUS
0 1.0.90.106                            56 61 6ms
1 1.0.90.106                            56 61 1ms
2 1.0.90.106                            56 61 1ms
3 1.0.90.106                            56 61 2ms
4 1.0.90.106                            56 61 1ms
5 1.0.90.106                            56 61 1ms
6 1.0.90.106                            56 61 1ms
7 1.0.90.106                            56 61 1ms
sent=8 received=8 packet-loss=0% min-rtt=1ms avg-rtt=1ms max-rtt=6ms

[admin@MikroTik] > ping 10.202.159.250
SEQ HOST                                SIZE TTL TIME  STATUS
0 10.202.159.250                        56 255 0ms
1 10.202.159.250                        56 255 0ms
2 10.202.159.250                        56 255 0ms
sent=3 received=3 packet-loss=0% min-rtt=0ms avg-rtt=0ms max-rtt=0ms

[admin@MikroTik] > ping 10.0.1.8
SEQ HOST                                SIZE TTL TIME  STATUS
0 10.0.1.8                              56 63 1ms
1 10.0.1.8                              56 63 1ms
sent=2 received=2 packet-loss=0% min-rtt=1ms avg-rtt=1ms max-rtt=1ms
```

Ping vers le leaf du réseau interne du groupe de Mathis : ce test confirme que le MikroTik est bien capable d'atteindre les équipements internes des autres groupes, prouvant que le routage intergroupe est fonctionnel.

Ping vers l'adresse IP du Catalyst du groupe de Mathias : ce test démontre que le BGP est correctement configuré et que les routes échangées permettent la communication entre les équipements de chaque groupe.

Ping vers notre propre leaf : ce test permet de vérifier que le protocole OSPF est bien opérationnel au sein de notre réseau interne, et que la connectivité est assurée entre le MikroTik et nos équipements via OSPF.

7. Backup du Mikrotik

Nous avons régulièrement effectué des backup du mikrotik pour éviter des problèmes de pertes de données pour X raisons. Pour cela, nous nous sommes rendus dans Files puis Backup:

	▲ File Name	Type	Size	Creation Time	
-	📄 backup_du_jeudi12.backup	backup	28.1 KiB	Jan/10/1970 23:57:50	Download

Verification des leafs & spines

```
spine01#sh ip bgp summary
BGP summary information for VRF default
Router identifier 10.255.0.1, local AS number 64555
Neighbor Status Codes: m - Under maintenance
```

Neighbor	V AS	MsgRcvd	MsgSent	InQ	OutQ	Up/Down	State	PfxRcd	PfxAcc
192.168.93.10	4 64555	462	295	0	0	04:00:59	Estab	80	80

```
spine01#sh ip ospf neighbor
```

Neighbor ID	Instance	VRF	Pri	State	Dead Time	Address	Interface
10.255.0.11	1	default	1	FULL/DR	00:00:33	10.0.1.0	Ethernet1
10.255.0.12	1	default	1	FULL/DR	00:00:37	10.0.1.2	Ethernet2
10.255.0.13	1	default	1	FULL/DR	00:00:36	10.0.1.4	Ethernet3

```
spine01#sh running-config | section bgp
router bgp 64555
  neighbor 192.168.93.10 remote-as 64555
  neighbor 192.168.93.10 update-source 192.168.93.9
  redistribute connected
  redistribute ospf
router ospf 1
  redistribute bgp
```

```
spine01#sh running-config | section ospf
router bgp 64555
  redistribute ospf
router ospf 1
  redistribute bgp
  redistribute connected
  network 10.0.1.0/31 area 0.0.0.0
  network 10.0.1.2/31 area 0.0.0.0
  network 10.0.1.4/31 area 0.0.0.0
  max-lsa 12000
```

```
spine02#sh ip bgp summary
BGP summary information for VRF default
Router identifier 10.255.0.2, local AS number 64555
Neighbor Status Codes: m - Under maintenance
```

Neighbor	V AS	MsgRcvd	MsgSent	InQ	OutQ	Up/Down	State	PfxRcd	PfxAcc
192.168.55.2	4 64555	47294	298	0	0	04:03:24	Estab	93	93

```
spine02#sh ip ospf neighbor
```

Neighbor ID	Instance	VRF	Pri	State	Dead Time	Address	Interface
10.255.0.13	1	default	1	FULL/DR	00:00:36	10.0.1.10	Ethernet3
10.255.0.12	1	default	1	FULL/DR	00:00:30	10.0.1.8	Ethernet2
10.255.0.11	1	default	1	FULL/DR	00:00:34	10.0.1.6	Ethernet1

```
spine02#sh running-config | section bgp
router bgp 64555
  neighbor 192.168.55.2 remote-as 64555
  neighbor 192.168.55.2 update-source 192.168.55.1
  redistribute connected
  redistribute ospf
router ospf 1
  redistribute bgp
spine02#sh running-config | section ospf
router bgp 64555
  redistribute ospf
router ospf 1
  redistribute bgp
  redistribute connected
  network 10.0.1.6/31 area 0.0.0.0
  network 10.0.1.8/31 area 0.0.0.0
  network 10.0.1.10/31 area 0.0.0.0
  max-lsa 12000
```

```
leaf01#sh ip ospf neighbor
Neighbor ID      Instance VRF      Pri State          Dead Time   Address     Interface
10.255.0.1       1       default  1   FULL/BDR        00:00:32   10.0.1.1   Ethernet1
10.255.0.2       1       default  1   FULL/BDR        00:00:32   10.0.1.7   Ethernet2
leaf01#sh running-config | section ospf
router ospf 1
 redistribute connected
 network 10.0.1.0/31 area 0.0.0.0
 network 10.0.1.6/31 area 0.0.0.0
 network 192.168.43.0/30 area 0.0.0.0
 network 192.168.78.0/30 area 0.0.0.0
 network 192.168.94.0/24 area 0.0.0.0
 network 192.168.95.0/24 area 0.0.0.0
 max-lsa 12000
```

```
leaf02#sh ip ospf neighbor
Neighbor ID      Instance VRF      Pri State          Dead Time   Address     Interface
10.255.0.1       1       default  1   FULL/BDR        00:00:29   10.0.1.3   Ethernet1
10.255.0.2       1       default  1   FULL/BDR        00:00:35   10.0.1.9   Ethernet2
leaf02#sh running-config | section ospf
router ospf 1
 redistribute connected
 network 10.0.1.2/31 area 0.0.0.0
 network 10.0.1.8/31 area 0.0.0.0
 network 192.168.43.0/30 area 0.0.0.0
 network 192.168.73.0/30 area 0.0.0.0
 max-lsa 12000
```

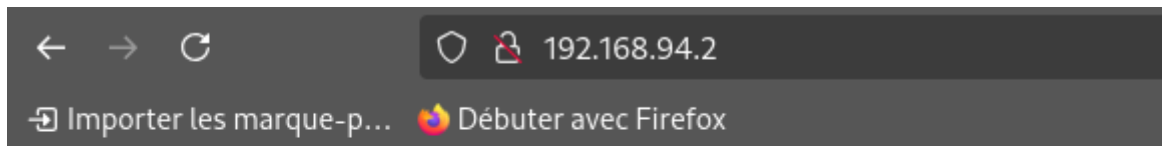
```
leaf03#sh ip ospf neighbor
Neighbor ID      Instance VRF      Pri State          Dead Time   Address     Interface
10.255.0.1       1       default  1   FULL/BDR        00:00:31   10.0.1.5   Ethernet1
10.255.0.2       1       default  1   FULL/BDR        00:00:38   10.0.1.11  Ethernet2
leaf03#sh running-config | section ospf
router ospf 1
 redistribute connected
 network 10.0.1.4/31 area 0.0.0.0
 network 10.0.1.10/31 area 0.0.0.0
 network 192.168.99.0/24 area 0.0.0.0
 max-lsa 12000
```

Ajout des micro services

Nous avons pu joindre notre serveur web en mettant une route vers l'adresse du réseau du **container web** sur la machine qui est dans la salle, puis nous avons fait un **curl** de l'ip. Nous voyons bien que nous avons accès au serveur web.

```
test@8-1:~$ sudo ip route add 192.168.94.0/24 via 10.202.8.1
test@8-1:~$ curl 192.168.94.2
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="UTF-8">
    <title>Bienvenue</title>
  </head>
  <body>
    <h1>Bienvenue chez la team yokoso</h1>
  </body>
</html>
test@8-1:~$ ip -br a
lo                UNKNOWN      127.0.0.1/8  ::1/128
enp0s31f6         UP          10.202.8.1/16 fe80::25d1:ffe
```

Nous y avons aussi accès par **interface web** (preuve plus esthétique) :



Bienvenue chez la team yokoso

Intégration VPN avec wireguard

Pour l'intégration du VPN nous avons opté pour Wireguard Easy une solution qui simplifie le démarrage de notre wireguard avec un container et qui va générer les conf prête a l'emploi pour le client.

- **Récupération de l'image:**

```
docker pull weejewel/wg-easy:latest
```

- **Ajout du docker compose**

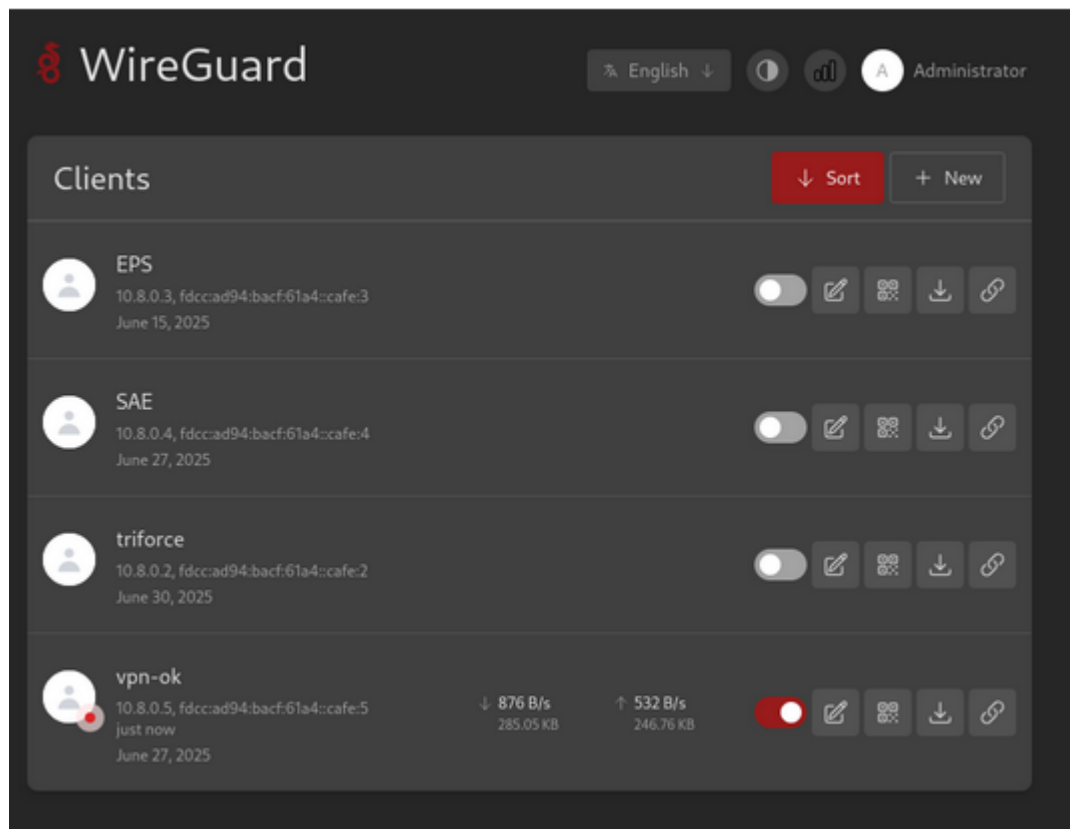
```
volumes:
  etc_wireguard:

services:
  wg-easy:
    environment:
      - PORT=51821
      - HOST=0.0.0.0
      - INSECURE=true
```

```
image: ghcr.io/wg-easy/wg-easy:15
container_name: wg-easy
networks:
wg:
  ipv4_address: 10.202.30.1
volumes:
- etc_wireguard:/etc/wireguard
- /lib/modules:/lib/modules:ro
ports:
- "51820:51820/udp"
- "51821:51821/tcp"
restart: unless-stopped
cap_add:
- NET_ADMIN
- SYS_MODULE
sysctls:
- net.ipv4.ip_forward=1
- net.ipv4.conf.all.src_valid_mark=1

networks:
  wg:
    driver: macvlan
    enable_ipv6: false
    driver_opts:
      parent: enp0s31f6
    ipam:
      config:
        - subnet: 10.202.0.0/16
          gateway: 10.202.10.1
```

Une fois le docker-compose up on peut verifier le bon fonctionnement de WireGuard sur l'interface web <http://10.202.30.1:51821> est confirmé par la présence du point rouge indiquant une connexion active, ainsi que par l'affichage des débits en émission et en réception.



Partie client

Il faut côté client simple faire creation du wg0.conf :

`nano wg0.conf`

```
[Interface]
PrivateKey = 00neb7ES0y06kfgS3ISj4I7MfAGwFIbMka4aBCi79F0=
Address = 10.8.0.5/24, fdcc:ad94:bacf:61a4::cafe:5/112
DNS = 1.1.1.1, 2606:4700:4700::1111
MTU = 1420

[Peer]
PublicKey = ua5pK/+TZK1H+XCd1/zDuek1JtHV19t4Zb2VGg1QcJg=
PresharedKey = fVxfg7BM0q3uFgDbxeaPg+KFtgz6C2/bj0KJU21z60=
AllowedIPs = 0.0.0.0/0, ::/0
PersistentKeepalive = 0
Endpoint = 10.202.30.1:51820
```

On peut verifier déjà son bon fonctionnement du peer avec la commande `wg show`

```

root@0-211:/etc/wireguard# wg show
interface: mathys
  public key: g/0UAogZOWJQ8uAQAv+n9tZ537R+yDZ4ap3EbTbj1zw=
  private key: (hidden)
  listening port: 52595
peer: CHa0r6PGYeCn+7CLCkxqbyn00Dzg76ubi/jRXKyuazc=
  preshared key: (hidden)
  endpoint: 10.202.214.134:51820
  allowed ips: 1.0.0.0/16
  latest handshake: 49 seconds ago
  transfer: 1.37 KiB received, 1.11 KiB sent

```

Le groupe de mathis, qui est depuis son datacenter, arrive a acceder aux serveur dns dans notre data center grace aux tunnel vpn wireguard :

```

bash-5.1# dig yokoso.com @192.168.95.2

; <<>> DiG 9.16.23-RH <<>> yokoso.com @192.168.95.2
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 7496
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;yokoso.com.                IN      A

;; ANSWER SECTION:
yokoso.com.                3600    IN      A      192.168.73.2

;; Query time: 4 msec
;; SERVER: 192.168.95.2#53(192.168.95.2)
;; WHEN: Fri Jun 13 08:01:25 UTC 2025
;; MSG SIZE rcvd: 55

bash-5.1# traceroute 192.168.95.2
traceroute to 192.168.95.2 (192.168.95.2), 30 hops max, 60 byte packets
 1 _gateway (172.16.34.1)  0.380 ms  0.410 ms  0.454 ms
 2 10.202.10.1 (10.202.10.1)  3.513 ms  3.512 ms  3.435 ms
 3 192.168.93.9 (192.168.93.9)  4.608 ms  4.484 ms  4.637 ms
 4 10.0.1.0 (10.0.1.0)  4.965 ms  4.676 ms  4.861 ms
 5 192.168.95.2 (192.168.95.2)  5.079 ms  4.660 ms  5.244 ms
bash-5.1#

```

Nous aussi accedons aux leurs grace a wireguard-easy :

```
root@0-211:/etc/wireguard# wg-quick up mathys
[#] ip link add mathys type wireguard
[#] wg setconf mathys /dev/fd/63
[#] ip -4 address add 10.34.134.3/24 dev mathys
[#] ip -6 address add fdcc:ad94:bacf:61a4::cafe:3/112 dev mathys
[#] ip link set mtu 1420 up dev mathys
[#] resolvconf -a tun.mathys -m 0 -x
[#] ip -4 route add 1.0.0.0/16 dev mathys
root@0-211:/etc/wireguard# curl 1.0.90.106
<html>
<head>
<title>Mathys LOPINTO LEAF 3</title>
</head>
<body>
<h1>Mathys LOPINTO LEAF 3</h1>
<h2>Test out amazing DNS service at 1.0.90.102</h2>
<p>*Only work for triforme.fr domain</p>
</body>
</html>

root@0-211:/etc/wireguard# dig +short mathys-l.triforme.fr @1.0.90.102
1.0.90.106
root@0-211:/etc/wireguard# dig mathys-l.triforme.fr @1.0.90.102

; <<>> DiG 9.18.33-1-deb12u2-Debian <<>> mathys-l.triforme.fr @1.0.90.102
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 30526
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 1232
;; COOKIE: 8a1a38384f3ffd4101000000684bd31240d536b4a5f6f69b (good)
;; QUESTION SECTION:
;mathys-l.triforme.fr.          IN      A

;; ANSWER SECTION:
mathys-l.triforme.fr.  86400  IN      A      1.0.90.106

;; Query time: 4 msec
;; SERVER: 1.0.90.102#53(1.0.90.102) (UDP)
;; WHEN: Fri Jun 13 09:28:18 CEST 2025
;; MSG SIZE rcvd: 93
```

Mise en place de la télémétrie avec gNMI

Objectif

Nous avons mis en place une solution de **télémétrie réseau avec gNMI** pour superviser/monitorer en temps réel les performances de mes routeurs **Arista cEOS** déployés dans une topologie **Leaf & Spine** avec Containerlab.

Le but était de collecter automatiquement des métriques réseau (ex : bgp ospf) et de les visualiser via une stack d'observabilité (Prometheus + Grafana + gnmic).

Étapes de mise en place

1. Activer le service gNMI sur chaque routeur Arista

Sur chaque cEOS, j'ai activé l'API gNMI en CLI comme ceci :

```
conf t
management api gnmi
  transport grpc default
  no shutdown
  username gnmiuser password MonMotDePasse123
```

2. Écrire le fichier gnmic.yaml

J'ai ensuite écrit un fichier de configuration gnmic.yaml pour connecter gnmic à tous mes routeurs et définir les métriques à collecter.

```
targets:

  leaf1:
    address: 172.20.20.3:6030
    username: gnmiuser
    password: MonMotDePasse123
    insecure: true

  leaf2:
    address: 172.20.20.6:6030
    username: gnmiuser
    password: MonMotDePasse123
    insecure: true

  leaf3:
    address: 172.20.20.4:6030
    username: gnmiuser
    password: MonMotDePasse123
    insecure: true

  spine1:
    address: 172.20.20.5:6030
    username: gnmiuser
    password: MonMotDePasse123
    insecure: true

  spine2:
    address: 172.20.20.2:6030
    username: gnmiuser
    password: MonMotDePasse123
    insecure: true

subscriptions:

  #Interface Ethernet1 état + compteurs
  ethernet1-state:
    paths:
```

```
- /interfaces/interface[name=Ethernet1]/state
- /interfaces/interface[name=Ethernet1]/state/counters
stream-mode: sample
sample-interval: 5s
```

#4. 5. BGP neighbors et état session

bgp-neighbors-state:

paths:

```
- /network-instances/network-
```

```
instance[name=default]/protocols/protocol[identifiant=BGP]
[name=BGP]/bgp/neighbors/neighbor
```

```
- /network-instances/network-
```

```
instance[name=default]/protocols/protocol[identifiant=BGP]
[name=BGP]/bgp/neighbors/neighbor/state/session-state
```

stream-mode: sample

sample-interval: 20s

format: prom

#6. Toutes les infos de l'équipement (system subtree)

system-info:

paths:

```
- /system
```

stream-mode: sample

sample-interval: 60s

#7. Etat mémoire

memory-state:

paths:

```
- /system/memory/state
```

stream-mode: sample

sample-interval: 15s

#8. Nombre de CPU en format flat

cpu-info-flat:

paths:

```
- /system/cpus/cpu
```

stream-mode: sample

sample-interval: 15s

output: flat

#10. Souscrire aux compteurs interface Ethernet1 toutes les 5s

ethernet1-counters-sub:

paths:

```
- /interfaces/interface[name=Ethernet1]/state/counters
```

stream-mode: sample

sample-interval: 5s

#11. Récupérer les routes BGP (attention à la hiérarchie précise)

bgp-routes:

paths:

```
- /network-instances/network-
```

```
instance[name=default]/protocols/protocol[identifiant=BGP]
```

```
[name=BGP]/bgp/rib/afi-safis/afi-safi/ipv4-unicast/loc-rib/routes/route
  stream-mode: sample
  sample-interval: 30s
  format: prom

#12. Processus et leur nombre
processes-info:
  paths:
    - /system/processes/process
  stream-mode: sample
  sample-interval: 30s

outputs:
  prometheus:
    type: prometheus
    listen: ":9804"

api-server:
  enable-metrics: true
  enable-pprof: true
  address: ":9804"

logging:
  level: info
  format: console
# Ensuite, j'ai configuré Prometheus pour récupérer les données de gnmic:
global:
  scrape_interval: 10s

scrape_configs:
  - job_name: 'gnmic'
    static_configs:
      - targets: ['10.202.0.121:9804']

# Lancer Prometheus gnmic et Grafana via Docker
```

J'ai ensuite utilisé un fichier docker-compose.yml pour lancer des dockers Prometheus gnmic ainsi Grafana ensemble:

```
version: '3.8'

services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9090:9090"
    networks:
      - monitor-net
```

```

grafana:
  image: grafana/grafana:latest
  container_name: grafana
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=admin
  volumes:
    - grafana_data:/var/lib/grafana
  networks:
    - monitor-net

gnmic:
  image: ghcr.io/openconfig/gnmic:latest
  container_name: gnmic
  volumes:
    - ./gnmic.yaml:/app/gnmic.yaml
  command: ["subscribe", "--config", "/app/gnmic.yaml"]
  ports:
    - "9804:9804"
  networks:
    - monitor-net
  depends_on:
    - prometheus

volumes:
  grafana_data:

networks:
  monitor-net:
    driver: bridge

```

Puis je lance le tout avec la commande suivante :

`docker-compose up -d`

```

ed66050125fb ghcr.io/openconfig/gnmic:latest "/app/gnmic subscrib..." 5 hours ago Up 5 hours 0.0.0.0:9804->9804/tcp, :::9804->9804/tcp gnmic
42381a4a9fff prom/prometheus:latest "/bin/prometheus --c..." 5 hours ago Up 5 hours 0.0.0.0:9090->9090/tcp, :::9090->9090/tcp prometheus
2e2926a9faf9 grafana/grafana:latest "/run.sh" 5 hours ago Up 5 hours 0.0.0.0:3000->3000/tcp, :::3000->3000/tcp grafana
dde5bbc1f472 registry.iutbeziers.fr/ceos:4.33.1F "bash -c '/mnt/flash..." 5 hours ago Up 5 hours 0.0.0.0:6035->6030/tcp, :::6035->6030/tcp clab-ceos-dc-leaf3
317c9dcf16de registry.iutbeziers.fr/ceos:4.33.1F "bash -c '/mnt/flash..." 5 hours ago Up 5 hours 0.0.0.0:6031->6030/tcp, :::6031->6030/tcp clab-ceos-dc-spine1
ed73aa6b84de registry.iutbeziers.fr/ceos:4.33.1F "bash -c '/mnt/flash..." 5 hours ago Up 5 hours 0.0.0.0:6034->6030/tcp, :::6034->6030/tcp clab-ceos-dc-leaf2
7a78d9d4a87d registry.iutbeziers.fr/ceos:4.33.1F "bash -c '/mnt/flash..." 5 hours ago Up 5 hours 0.0.0.0:6033->6030/tcp, :::6033->6030/tcp clab-ceos-dc-leaf1
094062291dda registry.iutbeziers.fr/ceos:4.33.1F "bash -c '/mnt/flash..." 5 hours ago Up 5 hours 0.0.0.0:6032->6030/tcp, :::6032->6030/tcp clab-ceos-dc-spine2

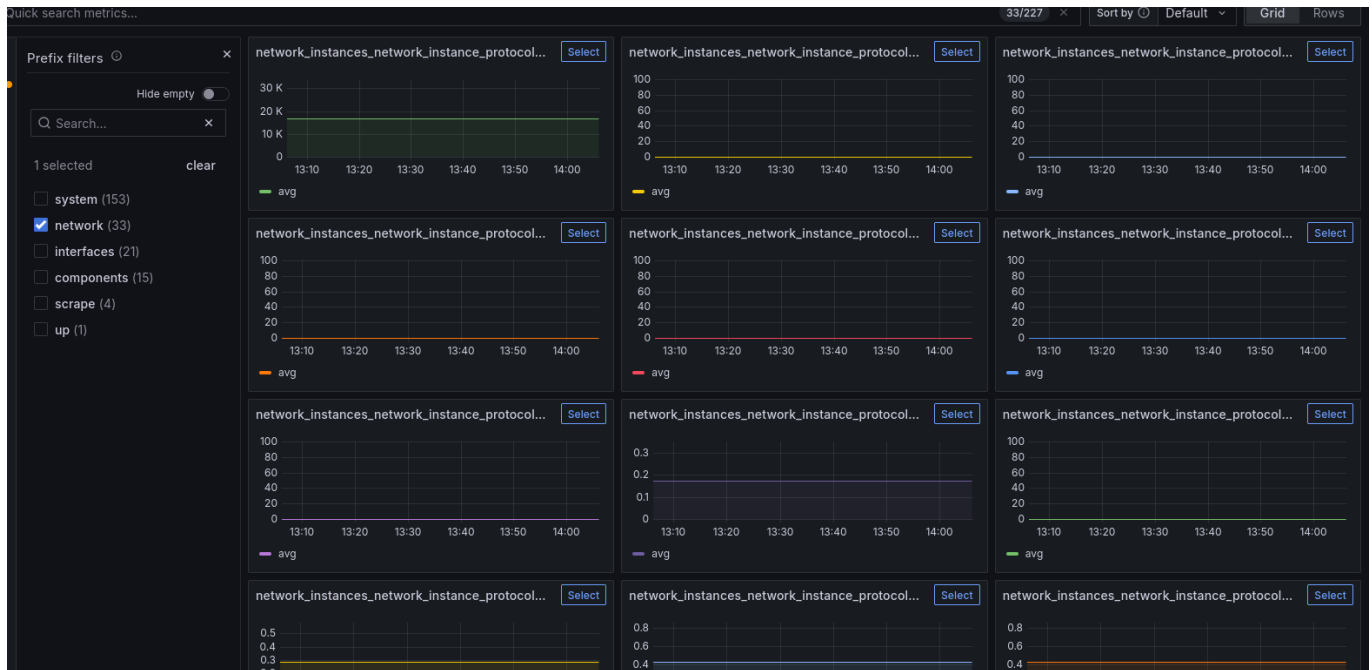
```

Ajouter Prometheus comme source de données dans Grafana

Une fois Grafana lancé (par défaut sur le port 3000), je me suis connecté à l'interface web :

URL : `http://localhost:3000` Login : admin / admin

Dans Grafana : Je vais dans "Settings > Data Sources" Je clique sur "Add data source" Je choisis Prometheus je rentre l'URL : `http://prometheus:9090` apres save et test et nous avons acces a nos metric



Endpoint	Labels	Last scrape	State
http://10.202.0.121:9804/metrics	instance="10.202.0.121:9804" job="gnmic"	3.981s ago 97ms	UP

Pour envoyer des données il faut d'abord installer gnmic

A laide de la commande `curl -sSL`

`https://raw.githubusercontent.com/openconfig/gnmic/main/install.sh | bash`

Puis pour envoyer les données il faut exécuter cette commande : `gnmic --config gnmic.yaml subscribe -d`

```
uint_val:1106}}}}
2025/06/12 19:20:36.694127 /home/runner/work/gnmic/gnmic/pkg/app/collector.go:71: [gnmic] target "spine2": gNMI Subscribe Response: &{SubscriptionName:ethernet1-counters-sub Subsc
fig:{"name":"ethernet1-counters-sub","paths":["/interfaces/interface[name=Ethernet1]/state/counters"],"mode":"stream","stream-mode":"sample","sample-interval":5000000000} Response
timestamp:1749746904000786576 prefix:{elem:{name:"interfaces"} elem:{name:"interface" key:{key:"name" value:"Ethernet1"}} elem:{name:"state"} elem:{name:"counters"}} update:
m:{name:"in-multicast-pkts"}} val:{uint_val:3826}} update:{path:{elem:{name:"in-octets"}} val:{uint_val:564573}} update:{path:{elem:{name:"in-pkts"}} val:{uint_val:4929}}}}
2025/06/12 19:20:36.707379 /home/runner/work/gnmic/gnmic/pkg/app/collector.go:71: [gnmic] target "leaf3": gNMI Subscribe Response: &{SubscriptionName:system-info SubscriptionConf
"system-info","paths":["/system"],"mode":"stream","stream-mode":"sample","sample-interval":60000000000} Response:update:{timestamp:1749746678776731771 update:{path:{elem:{name:"s
lem:{name:"cpus"} elem:{name:"cpu" key:{key:"index" value:"11"}} elem:{name:"state"} elem:{name:"idle"} elem:{name:"max-time"}} val:{uint_val:1749746678777560949}}}}

```

On peut verifier si nous avons bien recus les données/métrics sur le port 9804 correspondant au port de gnmic :

```

# HELP components_component_chassis_alarms_critical_state_status gNMIC generated metric
# TYPE components_component_chassis_alarms_critical_state_status untyped
components_component_chassis_alarms_critical_state_status{component_name="chassis-hostName",source="leaf1",subscription_name="equipment-capabilities"} 0
components_component_chassis_alarms_critical_state_status{component_name="chassis-hostName",source="leaf2",subscription_name="equipment-capabilities"} 0
components_component_chassis_alarms_critical_state_status{component_name="chassis-hostName",source="leaf3",subscription_name="equipment-capabilities"} 0
components_component_chassis_alarms_critical_state_status{component_name="chassis-hostName",source="spine1",subscription_name="equipment-capabilities"} 0
components_component_chassis_alarms_critical_state_status{component_name="chassis-hostName",source="spine2",subscription_name="equipment-capabilities"} 0
# HELP components_component_cpu_utilization_state_avg gNMIC generated metric
# TYPE components_component_cpu_utilization_state_avg untyped
components_component_cpu_utilization_state_avg{component_name="CPU0",source="leaf1",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU0",source="leaf2",subscription_name="equipment-capabilities"} 2
components_component_cpu_utilization_state_avg{component_name="CPU0",source="leaf3",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU0",source="spine1",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU0",source="spine2",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU1",source="leaf1",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU1",source="leaf2",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU1",source="leaf3",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU1",source="spine1",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU1",source="spine2",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU10",source="leaf1",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU10",source="leaf2",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU10",source="leaf3",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU10",source="spine1",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU10",source="spine2",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU11",source="leaf1",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU11",source="leaf2",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU11",source="leaf3",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU11",source="spine1",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU11",source="spine2",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU12",source="leaf1",subscription_name="equipment-capabilities"} 5
components_component_cpu_utilization_state_avg{component_name="CPU12",source="leaf2",subscription_name="equipment-capabilities"} 4
components_component_cpu_utilization_state_avg{component_name="CPU12",source="leaf3",subscription_name="equipment-capabilities"} 5
components_component_cpu_utilization_state_avg{component_name="CPU12",source="spine1",subscription_name="equipment-capabilities"} 4
components_component_cpu_utilization_state_avg{component_name="CPU12",source="spine2",subscription_name="equipment-capabilities"} 4
components_component_cpu_utilization_state_avg{component_name="CPU13",source="leaf1",subscription_name="equipment-capabilities"} 4
components_component_cpu_utilization_state_avg{component_name="CPU13",source="leaf2",subscription_name="equipment-capabilities"} 4
components_component_cpu_utilization_state_avg{component_name="CPU13",source="leaf3",subscription_name="equipment-capabilities"} 4
components_component_cpu_utilization_state_avg{component_name="CPU13",source="spine1",subscription_name="equipment-capabilities"} 4
components_component_cpu_utilization_state_avg{component_name="CPU13",source="spine2",subscription_name="equipment-capabilities"} 4
components_component_cpu_utilization_state_avg{component_name="CPU14",source="leaf1",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU14",source="leaf2",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU14",source="leaf3",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU14",source="spine1",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU14",source="spine2",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU15",source="leaf1",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU15",source="leaf2",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU15",source="leaf3",subscription_name="equipment-capabilities"} 3
components_component_cpu_utilization_state_avg{component_name="CPU15",source="spine1",subscription_name="equipment-capabilities"} 3

```

Dockerfile FRRouting (FRR)

Nous créons un Dockerfile permettant de construire une image Docker basée sur FRRouting (FRR) c.a.d routage dynamique pour BGP et OSPF.

2. Description de ce que j'ai fait dans le Dockerfile

```

FROM frrouting/frr:latest

RUN sed -i 's/bgpd=no/bgpd=yes/' /etc/frr/daemons && \
    sed -i 's/zebra=no/zebra=yes/' /etc/frr/daemons && \
    chown frr:frr /etc/frr/*

EXPOSE 179/tcp
EXPOSE 2601/tcp
EXPOSE 2604/tcp

```

Base de l'image : Nous avons utilisé l'image officielle frrouting/frr:latest qui contient déjà une installation complète de FRR.

Activation des daemons nécessaires : Par défaut, FRR désactive certains services. J'ai modifié le fichier `/etc/frr/daemons` pour activer :

bgpd (le démon BGP), qui permet la gestion des sessions BGP.

zebra, qui est le démon central gérant la table de routage.

Exposition des ports nécessaire : **179/TCP** pour bgp et **2061(et4)/TCP** pour FRR (vty)

4. Deploiement Dockerfile.frr

```
test@0-179:~/Documents/YOKOSO/devcloud-lab$ docker build -t frr-custom -f Dockerfile.frr .
Sending build context to Docker daemon 69.63kB
Step 1/5 : FROM frrouting/frr:latest
--> d19bacb84eae
Step 2/5 : RUN sed -i 's/bgpd=no/bgpd=yes/' /etc/frr/daemons && sed -i 's/zebra=no/zebra=yes/' /etc/frr/daemons && chown frr:frr /etc/frr/*
--> Using cache
--> d93007ac19ea
Step 3/5 : EXPOSE 179/tcp
--> Using cache
--> be89cc8d4b40
Step 4/5 : EXPOSE 2601/tcp
--> Using cache
--> ae0660c94d71
Step 5/5 : EXPOSE 2604/tcp
--> Using cache
--> 0022676b9de6
Successfully built 0022676b9de6
Successfully tagged frr-custom:latest
```

5 .Déploiement de la topologie avec Containerlab :

Après avoir construit et lancé cette image, il faudra créer un fichier YAML décrivant la topologie réseau, par exemple `leaf-spine.clab.yml`.

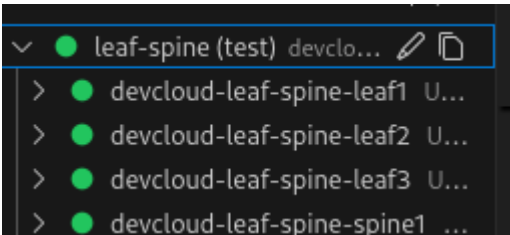
```
1  name: leaf-spine
2
3  prefix: devcloud
4
5  topology:
6    nodes:
7      leaf1:
8        kind: linux
9        image: frr-custom
10       startup-config: configs/leaf1/frr.conf
11      leaf2:
12        kind: linux
13        image: frr-custom
14        startup-config: configs/leaf2/frr.conf
15      leaf3:
16        kind: linux
17        image: frr-custom
18        startup-config: configs/leaf3/frr.conf
19      spine1:
20        kind: linux
21        image: frr-custom
22        startup-config: configs/spine1/frr.conf
23      spine2:
24        kind: linux
25        image: frr-custom
26        startup-config: configs/spine2/frr.conf
27
28    links:
29      - endpoints: ["leaf1:eth1", "spine1:eth1"]
30      - endpoints: ["leaf1:eth2", "spine2:eth1"]
31      - endpoints: ["leaf2:eth1", "spine1:eth2"]
32      - endpoints: ["leaf2:eth2", "spine2:eth2"]
33      - endpoints: ["leaf3:eth1", "spine1:eth3"]
34      - endpoints: ["leaf3:eth2", "spine2:eth3"]
35
36
```

Pour déployer la topologie, il suffit ensuite de lancer la commande : `sudo containerlab deploy -t leaf-spine.clab.yml`

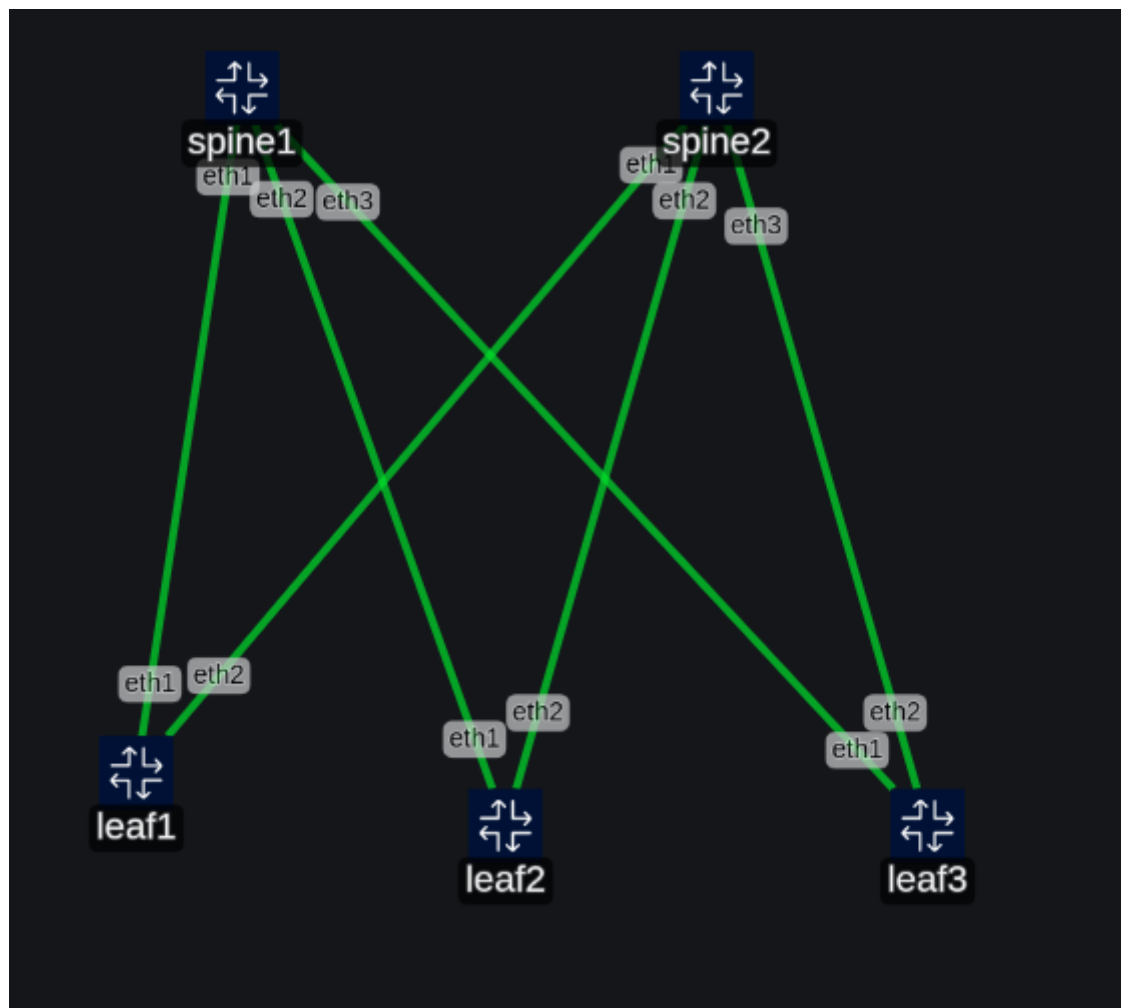
```
test@0-179:~/Documents/YOKOSO/devcloud-lab$ sudo containerlab deploy -t leaf_spine_bgp_lab.yml
[sudo] Mot de passe de test :
16:07:38 INFO Containerlab started version=0.68.0
16:07:38 INFO Parsing & checking topology file=leaf_spine_bgp_lab.yml
16:07:38 INFO Creating lab directory path=/home/test/Documents/YOKOSO/devcloud-lab/clab-leaf-spine
16:07:38 INFO Creating container name=leaf3
16:07:38 INFO Creating container name=leaf1
16:07:38 INFO Creating container name=leaf2
16:07:38 INFO Creating container name=spine2
16:07:38 INFO Creating container name=spine1
16:07:39 INFO Created link: leaf2:eth1 --- spine1:eth2
16:07:39 INFO Created link: leaf1:eth1 --- spine1:eth1
16:07:40 INFO Created link: leaf3:eth1 --- spine1:eth3
16:07:40 INFO Created link: leaf1:eth2 --- spine2:eth1
16:07:40 INFO Created link: leaf2:eth2 --- spine2:eth2
16:07:40 INFO Created link: leaf3:eth2 --- spine2:eth3
16:07:40 INFO Adding host entries path=/etc/hosts
16:07:40 INFO Adding SSH config for nodes path=/etc/ssh/ssh_config.d/clab-leaf-spine.conf
```

Name	Kind/Image	State	IPv4/6 Address
devcloud-leaf-spine-leaf1	linux frr-custom	running	172.20.20.10 3fff:172:20:20::a
devcloud-leaf-spine-leaf2	linux frr-custom	running	172.20.20.8 3fff:172:20:20::8
devcloud-leaf-spine-leaf3	linux frr-custom	running	172.20.20.9 3fff:172:20:20::9
devcloud-leaf-spine-spine1	linux frr-custom	running	172.20.20.7 3fff:172:20:20::7
devcloud-leaf-spine-spine2	linux frr-custom	running	172.20.20.11 3fff:172:20:20::b

Cette commande va automatiquement déployer les containers basés sur l'image FRR configurée et interconnecter les routeurs selon la topologie définie.



Voici la topologie (topoviewer) disponible a l'aide de Containerlab(VsCode)



Mise en place du bgp dans chaque router:

```

Configurations BGP
Spines (route-reflectors)
Spine1 (172.20.20.7)
configure terminal
router bgp 64555
  bgp router-id 1.1.1.1
  bgp cluster-id 1.1.1.1
  bgp log-neighbor-changes
  neighbor 172.20.20.9 remote-as 64555
  neighbor 172.20.20.9 route-reflector-client
  neighbor 172.20.20.8 remote-as 64555
  neighbor 172.20.20.8 route-reflector-client
  neighbor 172.20.20.10 remote-as 64555
  neighbor 172.20.20.10 route-reflector-client
  address-family ipv4 unicast
    neighbor 172.20.20.9 activate
    neighbor 172.20.20.8 activate
    neighbor 172.20.20.10 activate
  exit-address-family
end
write

```

```
Spine2 (172.20.20.11)
configure terminal
router bgp 64555
  bgp router-id 1.1.1.2
  bgp cluster-id 1.1.1.2
  bgp log-neighbor-changes
  bgp log-neighbor-changes
  neighbor 172.20.20.9 remote-as 64555
  neighbor 172.20.20.9 route-reflector-client
  neighbor 172.20.20.8 remote-as 64555
  neighbor 172.20.20.8 route-reflector-client
  neighbor 172.20.20.10 remote-as 64555
  neighbor 172.20.20.10 route-reflector-client
  address-family ipv4 unicast
    neighbor 172.20.20.9 activate
    neighbor 172.20.20.8 activate
    neighbor 172.20.20.10 activate
  exit-address-family
end
write
```

```
Leafs (leaf1, leaf2, leaf3)
Leaf1 (172.20.20.9)
configure terminal
router bgp 64555
  bgp router-id 1.1.1.3
  bgp log-neighbor-changes
  neighbor 172.20.20.7 remote-as 64555
  neighbor 172.20.20.11 remote-as 64555
  address-family ipv4 unicast
    neighbor 172.20.20.7 activate
    neighbor 172.20.20.11 activate
  exit-address-family
end
write
```

```
Leaf2 (172.20.20.8)
configure terminal
router bgp 64555
  bgp router-id 1.1.1.4
  bgp log-neighbor-changes
  neighbor 172.20.20.7 remote-as 64555
  neighbor 172.20.20.11 remote-as 64555
  address-family ipv4 unicast
    neighbor 172.20.20.7 activate
    neighbor 172.20.20.11 activate
  exit-address-family
end
write
```

```

Leaf3 (172.20.20.10)
configure terminal
router bgp 64555
  bgp router-id 1.1.1.5
  bgp log-neighbor-changes
  neighbor 172.20.20.7 remote-as 64555
  neighbor 172.20.20.11 remote-as 64555
  address-family ipv4 unicast
    neighbor 172.20.20.7 activate
    neighbor 172.20.20.11 activate
  exit-address-family
end
write

```

Le BGP est bien mis en place et le ping aux autres routeurs fonctionne également en témoigne les screens ci dessous :

```

spine1# sh ip bgp summary

IPv4 Unicast Summary (VRF default):
BGP router identifier 1.1.1.1, local AS number 64555 vrf-id 0
BGP table version 0
RIB entries 0, using 0 bytes of memory
Peers 3, using 2151 KiB of memory

Neighbor      V      AS  MsgRcvd  MsgSent  TblVer  InQ  OutQ  Up/Down  State/PfxRcd  PfxSnt  Desc
172.20.20.8   4      64555    3        3        0    0    0 00:00:25      0           0 N/A
172.20.20.9   4      64555    3        3        0    0    0 00:00:39      0           0 N/A
172.20.20.10  4      64555    3        3        0    0    0 00:00:09      0           0 N/A

```

```

spine1# ping 172.20.20.9
PING 172.20.20.9 (172.20.20.9): 56 data bytes
64 bytes from 172.20.20.9: seq=0 ttl=64 time=0.065 ms
64 bytes from 172.20.20.9: seq=1 ttl=64 time=0.045 ms
^C

```

Vérification haute dispo pour infrastructure

Nous avons testé si, un spine tombe, l'infrastructure tiens.

Avant de faire tomber le spine 2, nous avons depuis le leaf 3 effectué un traceroute vers le catalyst du groupe de Mathias:

```

leaf03#traceroute 10.202.159.250
traceroute to 10.202.159.250 (10.202.159.250), 30 hops max, 60 byte packets
 1  10.0.1.5 (10.0.1.5)  0.048 ms  0.011 ms  0.010 ms
 2  192.168.55.2 (192.168.55.2)  3.162 ms  3.256 ms  3.263 ms
 3  10.202.159.250 (10.202.159.250)  5.386 ms  *  *
leaf03#

```

Nous voyons que le traceroute passe par le Spine 2 (l'IP **192.168.55.2**)

Maintenant, nous désactivons **toutes les interfaces** du Spine 2:

```
test@0-211:~/Documents/YOKOSO/prototype$ docker exec -it clab-ceos-dc-min-spine2 Cli
spine02>en
spine02#sh ip interface brief
```

Interface	IP Address	Status	Protocol	MTU	Address Owner
Ethernet1	10.0.1.7/31	up	up	1500	
Ethernet2	10.0.1.9/31	up	up	1500	
Ethernet3	10.0.1.11/31	up	up	1500	
Ethernet4	192.168.55.1/30	up	up	1500	
Loopback0	10.255.0.2/32	up	up	65535	
Management0	172.20.20.6/24	up	up	1500	

```

spine02#conf t
spine02(config)#interface ethernet 1
spine02(config-if-Et1)#shut
spine02(config-if-Et1)#exit
spine02(config)#interface ethernet 2
spine02(config-if-Et2)#shut
spine02(config-if-Et2)#exit
spine02(config)#interface ethernet 3
spine02(config-if-Et3)#shut
spine02(config-if-Et3)#exit
spine02(config)#interface ethernet 4
spine02(config-if-Et4)#shut
spine02(config-if-Et4)#exit
spine02(config)#interface loopback 0
spine02(config-if-Lo0)#shut
spine02(config-if-Lo0)#exit
spine02(config)#interface management 0
spine02(config-if-Ma0)#shut
spine02(config-if-Ma0)#exit
spine02(config)#end
spine02#wr
Copy completed successfully.
spine02#exit
test@0-211:~/Documents/YOKOSO/prototype$ docker exec -it clab-ceos-dc-min-leaf3 Cli
leaf03>en
leaf03#traceroute 10.202.159.250
traceroute to 10.202.159.250 (10.202.159.250), 30 hops max, 60 byte packets
 1  10.0.1.5 (10.0.1.5)  0.046 ms  0.013 ms  0.008 ms
 2  192.168.93.10 (192.168.93.10)  3.228 ms  3.346 ms  3.361 ms
 3  * * 10.202.159.250 (10.202.159.250)  3.938 ms

```

Et comme vous pouvez voir sur la capture d'écran, à la dernière commande, le traceroute passe par le Spine 1 (l'ip **192.168.93.10**):

```
192.168.93.10 (192.168.93.10)  1.892 ms  1.918 ms  2.092 ms
```

Cela prouve bien la haute disponibilité de notre architecture.

HA-PROXY

Nous avons choisi HA Proxy pour assurer de la haute disponibilité pour l'accès de notre serveur web. Si l'un tombe, un autre prend le relais.

Voici le fichier de configuration avec en bas renseigné les deux IP des serveur web qui doivent être joints:

```
be > configs > ⚙ haproxy.cfg
global
    log /dev/log local0
    maxconn 1000

defaults
    log      global
    mode     http
    option   httplog
    timeout  connect 5000
    timeout  client  50000
    timeout  server  50000

frontend http_front
    bind *:80
    default_backend http_back

backend http_back
    balance roundrobin
    server web1 192.168.78.2:80 check
    server web2 192.168.43.2:80 check
```

Pour vérifier son bon fonctionnement, sur une machine de la salle, nous avons fait un curl -I suivit de l'ip de HA-Proxy , et nous remarquons que a chaque curl nous avons une réponse un coup du serveur web 1 (nginx) et un coup du serveur web 2 (apache):

```
test@0-211:~$ curl -I 192.168.73.2
HTTP/1.1 200 OK
server: nginx/1.27.5
date: Fri, 13 Jun 2025 14:11:29 GMT
content-type: text/html
content-length: 169
last-modified: Fri, 13 Jun 2025 07:02:26 GMT
etag: "684bcd02-a9"
accept-ranges: bytes

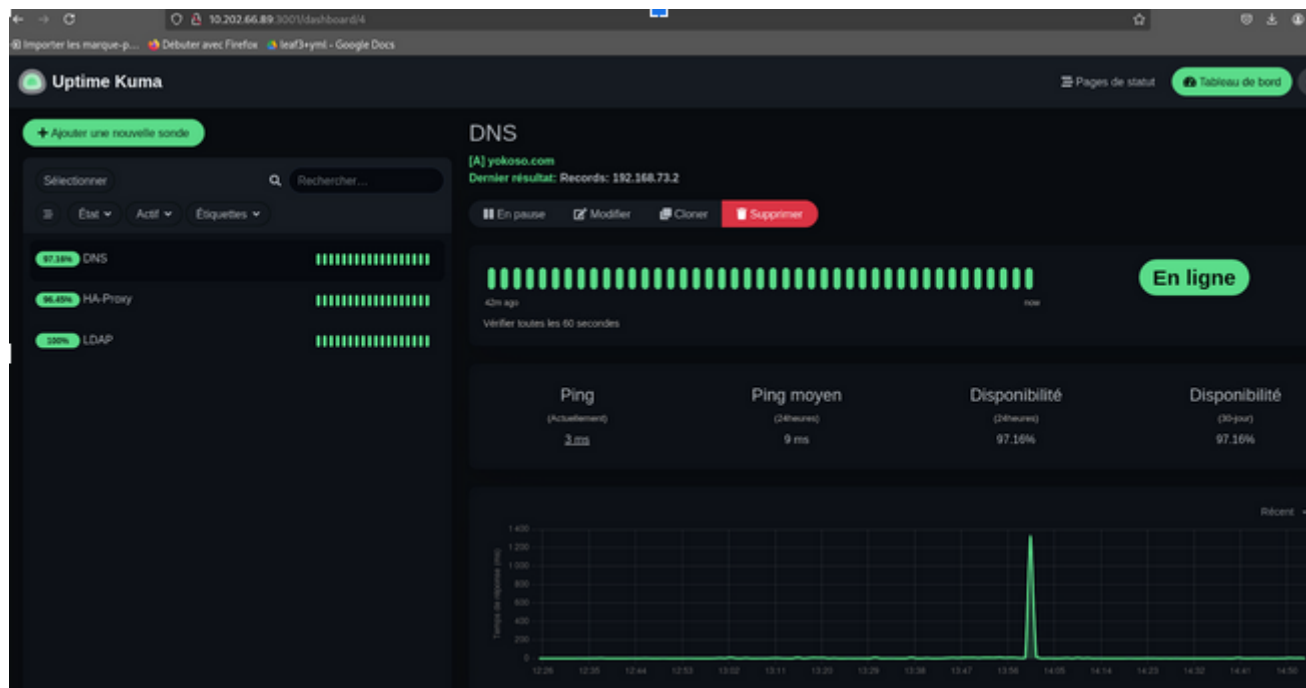
test@0-211:~$ curl -I 192.168.73.2
HTTP/1.1 200 OK
date: Fri, 13 Jun 2025 14:11:32 GMT
server: Apache/2.4.63 (Unix)
last-modified: Fri, 13 Jun 2025 07:02:26 GMT
etag: "a9-6376ea0f0e030"
accept-ranges: bytes
content-length: 169
content-type: text/html

test@0-211:~$ curl -I 192.168.73.2
HTTP/1.1 200 OK
server: nginx/1.27.5
date: Fri, 13 Jun 2025 14:11:33 GMT
content-type: text/html
content-length: 169
last-modified: Fri, 13 Jun 2025 07:02:26 GMT
etag: "684bcd02-a9"
accept-ranges: bytes
```

Uptime-Kuma

On a mis en place un docker uptime-kuma lié au réseau Leaf&Spine via le catalyst comme outil de supervision afin de surveiller l'état de mon infrastructure.

Le DNS, LDAP ainsi que notre HAProxy qui réunit plusieurs lien web en guise d'équilibreur de charge



On peut voir qu'actuellement ils tournent tous correctement

Problème Rencontré

Au cours de cette SAE, plusieurs difficultés ont été rencontrées. Tout d'abord, l'oubli de certaines routes a constitué un problème mineur mais aux conséquences importantes, rapidement corrigé après identification. Concernant le Catalyst, la VM Router a subi un freeze ; après plusieurs tentatives infructueuses de réinitialisation, il a finalement été nécessaire de changer de Catalyst pour poursuivre le projet. Une erreur de configuration a également été commise en plaçant deux bridges lan-br, ce qui a rendu le Catalyst inopérant. Malgré la suppression de l'un des bridges via le CLI, il n'a pas été possible de récupérer l'interface web, ce qui a conduit à une réinitialisation complète du matériel. Du côté du Mikrotik, le service DHCP attribuait initialement de mauvaises adresses IP, un problème qui a été rapidement résolu en retirant le service fautif. L'implémentation simultanée d'OSPF et de BGP a posé des difficultés, notamment concernant la redistribution des loopbacks qui n'a pas abouti comme prévu. Enfin, la partie télémétrie n'a été que partiellement réussie : la configuration a pu avancer jusqu'à l'utilisation de gnmic, et une solution potentielle a été identifiée pour aller plus loin.